

EJERCICIOS, PROBLEMAS, CASOS PRÁCTICOS



Universidad de Jaén

Escuela Politécnica Superior de Jaén

Ejercicios de introducción a la programación y problemas clásicos de sistemas concurrentes y distribuidos resueltos en Java

Francisco Jesús Martínez Mimbrera
David Díaz Jiménez

20/01/2026

Sistemas concurrentes y distribuidos



CREA

SISTEMAS CONCURRENTES Y DISTRIBUIDOS

Ejercicios de introducción a la programación y
problemas clásicos de Sistemas concurrentes y
distribuidos resueltos en Java



Universidad de Jaén

Escuela Politécnica Superior de Jaén
Departamento de Informática

Autores:

Francisco Jesús Martínez Mimbrera
David Díaz Jiménez

Título: Sistemas Concurrentes y Distribuidos: Ejercicios de introducción a la programación y problemas clásicos de Sistemas concurrentes y distribuidos resueltos en Java

Autores: Francisco J. Martínez Mimblera y David Díaz Jiménez

Versión: 1

Fecha: 20 de enero de 2026

Licencia:

Este manual se distribuye bajo una licencia **Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0)**.



Usted es libre de:

- **Compartir** - copiar y redistribuir el material en cualquier medio o formato.
- **Adaptar** - remezclar, transformar y construir a partir del material.

Bajo los siguientes términos:

- **Atribución** - Debe dar crédito de manera adecuada.
- **No Comercial** - No puede hacer uso del material con fines comerciales.
- **Compartir Igual** - Si remezcla, transforma o crea a partir del material, debe distribuir su contribución bajo la misma licencia del original.

AVISO IMPORTANTE SOBRE DERECHOS DE TERCEROS:

Esta licencia afecta a los textos originales, códigos fuente y explicaciones creadas por el autor de este manual.

Exclusiones: Las fotografías, diagramas, logotipos y citas textuales de otros autores incluidos en esta obra conservan sus propios derechos de autor (Copyright) o licencias originales y se incluyen aquí con fines educativos y de citación académica. El uso de estos materiales por terceros fuera del contexto de este manual puede requerir el permiso de sus respectivos titulares.

Este manual ha sido desarrollado y maquetado utilizando $\text{\LaTeX}$.

Índice general

1. Introducción	5
2. E/S por teclado y pantalla, y operaciones aritméticas	9
2.1. Mi primer programa - Hola mundo!!	10
2.2. Saludo personalizado	10
2.3. Capturar distinto tipo de información desde teclado y mostrarla por pantalla	12
2.4. Calcular el área de cuadrado a partir de un lado	13
2.5. Conversor de pesetas a Euros	14
2.6. Suma, Resta, Multiplicación y División de dos números	15
2.7. Precio medio de un producto en función de los precios de éste en otros establecimientos	16
2.8. Suma de los “n” primeros números naturales consecutivos	17
2.9. Pasar de pulgadas a milímetros	18
2.10. Pasar de grados Celsius a grados Fahrenheit	19
2.11. Calcular estadísticas de suspensos, aprobados, notables y sobresalientes de la clase	20
2.12. Cálculo del precio de venta de un coche a partir del coste de fabricación, ganancia de la empresa e IVA	22
2.13. Calcular el perímetro de una circunferencia a partir del radio	23
2.14. Calcular la raíz cuadrada de un número dado	24
2.15. Calcular la raíz n-ésima de un número	25
2.16. Calcular el área de un triángulo rectángulo	26
2.17. Calcular el área de un triángulo equilátero	27
2.18. Calcular el área de cualquier triángulo a partir de sus lados mediante la fórmula de Herón	28
2.19. Calcular el volumen de un cilindro a partir de su altura y diámetro	29
2.20. Calcular el valor del lado “a” de un triángulo rectángulo dados el valor de la hipotenusa “h” y del lado “b”	31

2.21. Calcular la altura de un triángulo equilátero	32
2.22. Obtener el valor del binomio de suma al cuadrado $(a + b)^2$	33
3. Flujos de control, funciones, cadenas de caracteres, arrays y matrices	35
3.1. ¿Número par o impar?	36
3.2. Calcular si un número entero es múltiplo de 3 y a la vez par	37
3.3. Calcular si un año es bisiesto	38
3.4. Programa que calcule el Máximo Común Divisor de dos números enteros no negativos mediante el algoritmo de Euclides	39
3.5. Función para calcular del Máximo Común Divisor de dos números enteros no negativos mediante el algoritmo de Euclides	40
3.6. Función para calcular si un número entero es divisor de otro	41
3.7. Pasar a función el ejercicio propuesto número 2.18	42
3.8. Función para calcular el área de un circunferencia de radio válido	43
3.9. Calcular la suma de los “n” primeros números naturales (Con bucle)	45
3.10. Realizar un programa que sume números introducidos por teclado hasta que se escriba un número concreto de parada	46
3.11. Dados 10 números por teclado decir cuál es el mayor de todos utilizando una función	47
3.12. Calcular el tipo de triángulo en función de los lados	49
3.13. Resolver ecuaciones de segundo grado mediante la fórmula general (sólo soluciones reales)	51
3.14. Dados 10 números por teclado sumar los pares y los impares de manera separada y mostrar ambos resultados	53
3.15. Función para calcular el factorial de un número (iterativo)	54
3.16. Función para calcular el factorial de un número (recursivo)	56
3.17. Función para calcular el número combinatorio de N sobre M	58
3.18. Función para resolver $(a + b)^n$ utilizando el desarrollo del binomio de Newton	60
3.19. Implementar la función de Fibonacci para valores enteros positivos	62
3.20. Implementar un procedimiento para seleccionar y visualizar los K bits de orden inferior de un número entero proporcionado.	65
3.21. Implementar una función que invierta los últimos K bits de orden inferior de un número entero proporcionado	66
3.22. Invertir una cadena de texto leída por teclado, sin utilizar las funciones del propio lenguaje de programación	68
3.23. Leer una cadena de caracteres desde teclado, decir longitud y concatenar con otra utilizando las funciones de la biblioteca estándar de Java	69
3.24. Convertir cadenas a letras mayúsculas y contar sus vocales	70

3.25. Función para realizar la trasposición de una matriz de números enteros dada . . .	73
3.26. Función para multiplicar 2 matrices cuadradas de números enteros	74
4. Manejo de Ficheros y Memoria Dinámica	77
4.1. Crear y Escribir en un fichero	78
4.2. Leer y mostrar contenido de un fichero de texto	79
4.3. Contar caracteres y palabras de un fichero de texto	80
4.4. Guardar una estructura en un fichero binario y cargarla en memoria para ser mostrada por pantalla	81
4.5. Insertar al final de un fichero de texto	83
4.6. Asignación y liberación básica de memoria con vectores	84
4.7. Trabajo con Matrices Dinámicas 2D	85
4.8. Aumento del tamaño de un vector en tiempo de ejecución	87
5. Algoritmos de ordenación y búsqueda	89
5.1. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo de la Burbuja	90
5.2. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo Quicksort	91
5.3. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo Quicksort utilizando el método propio de Java para tipos primitivos . .	93
5.4. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo Mergesort	94
5.5. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo Shellsort	96
5.6. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo RadixSort	97
5.7. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo HeapSort	99
5.8. Dado un conjunto de números enteros, realizar la búsqueda de un número mediante el algoritmo de Búsqueda Lineal	101
5.9. Dado un conjunto de números enteros ordenados, realizar la búsqueda de un número mediante el algoritmo de Búsqueda Binaria	102
5.10. Dado un conjunto de números enteros no negativos ordenados, realizar la búsqueda de un número mediante el algoritmo de Búsqueda por Interpolación .	104
6. Trabajo con problemas Concurrentes y Distribuidos	107
6.1. Semáforos: Problema del productor-consumidor	108
6.2. Semáforos: Problema de los lectores escritores	111

6.3. Semáforos: Problema de la cena de los filósofos	115
6.4. Monitores: Problema del productor-consumidor	118
6.5. Monitores: Problema de los lectores escritores	122
6.6. Monitores: Problema de la cena de los filósofos	126
6.7. Paso de Mensajes Asíncrono. El Problema del productor consumidor	130
6.8. Paso de Mensajes Asíncrono: Problema de los lectores escritores	132
6.9. Paso de Mensajes Asíncrono: Problema de la cena de los filósofos	139
6.10. Paso de Mensajes Síncrono: El problema del productor consumidor	144
6.11. Paso de Mensajes Síncrono: Problema de los lectores escritores	146
6.12. Paso de Mensajes Síncrono: Problema de la cena de los filósofos	153

Bibliografía	158
---------------------	------------

Capítulo 1

Introducción

El propósito general de este manual es proveer a los alumnos de la asignatura de *Sistemas Concurrentes y Distribuidos*, perteneciente al Grado de Ingeniería Informática, de problemas resueltos básicos como introducción a Java y los Sistemas Concurrentes y Distribuidos. La idea es que dicho material pueda servir como apoyo al estudio de la asignatura.

El lenguaje de programación Java ha ido evolucionando a lo largo de los años desde su creación por James Gosling en 1991 en Sun Microsystems (siendo conocido al principio como *Oak* o *Green*) como un proyecto secreto. El primer documento oficial que hace referencia es el conocido como “white paper”, escrito por el creador James Gosling en 1995 [3]. La primera versión pública, JDK 1.0, fue lanzada por Sun Microsystems en 1996 [1][7]. La última versión LTS es la 25 [12]. Java es un lenguaje de programación de alto nivel, orientado a objetos y basado en clases independiente de la plataforma. Al contrario que C y C++, Java no está pensado para trabajar a tan bajo nivel y sí para entornos que exigen estabilidad, escalabilidad y alto rendimiento. Además, Java es un lenguaje seguro en memoria, comparado con C y C++, y cuenta con un recolector de basura (*Garbage Collector*) integrado.

Según el índice TIOBE¹, Java se sitúa como el 3º-4º lenguaje de programación más utilizado en el mundo en el año 2025, siempre en pugna con C++ y C.

En la computación moderna, la eficiencia ya no depende únicamente de la velocidad de un solo núcleo, sino de la capacidad de realizar múltiples tareas de forma simultánea. Podemos hablar de que un **sistema es concurrente** [21] cuando tiene la capacidad para gestionar múltiples tareas que empiezan, se ejecutan y se completan en intervalos de tiempo superpuestos. No necesariamente implica que se ejecuten al mismo tiempo (paralelismo), sino que el sistema gestiona el progreso de varias tareas de forma fluida. De la misma manera, podemos hablar de que un **sistema es distribuido** [21] cuando éste permite compartir recursos y distribuir la carga de trabajo entre diferentes nodos físicos conectados por una red .

¹<https://www.tiobe.com/tiobe-index/>

¿Por qué Java para Concurrencia y Distribución?

Java fue diseñado desde sus inicios con la red y la multitarea en mente, lo que le otorga bondades únicas para el aprendizaje y la implementación de estos sistemas [16]:

- **Soporte Nativo de Hilos (Threads):** Java integra la clase `Thread` y la interfaz `Runnable` en su núcleo, permitiendo que la creación de procesos ligeros sea sencilla y estandarizada.
- **Primitivas de Sincronización:** El lenguaje ofrece palabras clave como `synchronized` y mecanismos de alto nivel en el paquete `java.util.concurrent` (como semáforos, cerrojos y barreras) que facilitan la gestión de la exclusión mutua.
- **Modelo de Memoria Robusto:** El *Java Memory Model* (JMM) define reglas claras sobre cómo los hilos interactúan con la memoria, ayudando a prevenir condiciones de carrera.
- **Abstracción de Red:** Gracias a su API de red (`java.net`), implementar sistemas distribuidos mediante *sockets* es mucho más directo que en lenguajes que requieren librerías externas complejas.
- **Portabilidad:** La premisa *"Write Once, Run Anywhere"* es vital en sistemas distribuidos donde los nodos pueden tener arquitecturas de hardware diferentes.

Para la elaboración de este libro se han utilizado las versiones más recientes del lenguaje y de los compiladores.

Estructura del manual

Para alcanzar el propósito planteado, este manual se va a estructurar en los siguientes capítulos para una mejor comprensión por parte del lector.

- **Capítulo 2. E/S por teclado y pantalla, y operaciones aritméticas.** En este capítulo pretendemos realizar ejercicios básicos de toma de contacto con el lenguaje y los primeros trabajos con las entradas de información por teclado y las salidas por pantalla. Asimismo, se verá cómo resolver problemas que implican operaciones aritméticas básicas.
- **Capítulo 3. Flujos de control, funciones, cadenas de caracteres, arrays y matrices.** En este capítulo se van a presentar ejercicios relativos al trabajo con estructuras de flujos de control, trabajo con funciones y procedimientos, cadenas de caracteres, arrays y matrices.
- **Capítulo 4. Manejo de Ficheros y Memoria Dinámica.** Este capítulo lo dedicaremos a resolver problemas de manejo de ficheros para guardar información y problemas que trabajen con estructuras de datos dinámicas.

-
- **Capítulo 5. Algoritmos de ordenación y búsqueda.** Como parte interesante de la resolución de problemas básicos de programación, siempre está bien el tener conocimiento de algoritmos de ordenación y búsqueda que nos ayuden a resolver problemas simples que sin dichos algoritmos su tiempo de ejecución se volvería inabarcable.
 - **Capítulo 6. Trabajo con problemas concurrentes y distribuidos.** En este capítulo vamos a presentar problemas típicos de concurrencia, los cuales serán resueltos por diferentes técnicas de las más relevantes en la literatura.

Finalmente, el manual concluye con una **Recopilación Bibliográfica** de las fuentes más destacadas para repasar conceptos y sintaxis de programación en Java.

Herramientas de trabajo para programar en Java

Para la elaboración de los problemas de este manual, se han utilizado la herramienta java incluida en el paquete OpenJDK 21 LTS [11] y el IDE IntelliJ IDEA [5]. Ambos están disponibles para los tres sistemas operativos más importantes del mercado, Windows, macOS y Linux.

Para ejecutar los problemas que se encuentran en el manual, en primer lugar se debe de crear el fichero.java con el problema a resolver y posteriormente, hay dos formas de poder ejecutarlos: Forma 1: Ejecutando directamente la herramienta java, la cual crea una copia en memoria del fichero .class y ejecuta el código directamente sobre la JVM. Forma 2: Ejecutar la herramienta javac y crear los .class y posteriormente ejecutar la herramienta java sobre los ficheros .class para lanzar el ejecutable en la JVM. También, el lector puede optar por descargar directamente el fichero.java del repositorio GitHub habilitado para ello y posteriormente elegir alguna de las formas de ejecución comentadas. Esta operación se puede hacer tanto en la terminal utilizando los comandos “java fichero.java” o “javac fichero.java” y después “java fichero” como desde un IDE como VSC [10] o IntelliJ IDEA [5].

Código fuente de los ejemplos

Para que el lector pueda utilizar más cómodamente los ejemplos de este manual, éstos han sido subidos a un repositorio público para que puedan ser descargados. Los ficheros.java están codificados con el juego de caracteres UTF-8. En la siguiente dirección se podrán descargar los problemas resueltos.

<https://github.com/franmartinezmimbrera/problemasjava>

Capítulo 2

E/S por teclado y pantalla, y operaciones aritméticas

En este capítulo se van a resolver una serie de problemas utilizando los distintos tipos de datos, operadores y expresiones en los lenguajes propuestos. Para estos problemas, las entradas de datos las realizaremos mediante el teclado o con precarga de variables, mientras que las salidas se realizarán por pantalla. Los ejercicios resueltos que contiene nos van a servir de base para familiarizarnos con las operaciones básicas que se suelen realizar en un lenguaje de programación, así como para empezar a utilizar algunas librerías externas tales como la librería para ciertas operaciones matemáticas. Como hemos resaltado en el capítulo de introducción, este manual no contiene teoría ni especificaciones sobre el lenguaje de programación, mostrando únicamente soluciones a problemas propuestos. Para revisar la teoría y especificaciones propias del lenguaje para E/S por teclado y pantalla y, operaciones aritméticas, proponemos al lector revisar algunos de los libros más destacados en la bibliografía tales como [16] [6] [14].

A continuación, presentamos los ejercicios resueltos para este capítulo.

2.1. Mi primer programa - Hola mundo!!

Este es el primer ejercicio/contacto de un programador con cualquier lenguaje de programación. Realizar un programa que muestre hola mundo!! por pantalla.

```
// fichero HolaMundo.java
// Este programa muestra el saludo hola mundo por pantalla.

public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("Hola mundo");
    }
}
```

En Java 25 es completamente funcional lo que hemos escrito anteriormente (la forma tradicional), pero desde la versión 21 existe una forma aún más concisa sin la necesidad explícita de la clase contenedora ni del método main (usando “Unnamed Classes and Instance Main Methods” en una vista previa¹, o simplemente ejecutando el código fuente). Esta opción hay que indicársela para compilar. Nosotros en esta colección de ejercicios vamos a trabajar con la forma tradicional y que es compatible con todas las versiones de java.

2.2. Saludo personalizado

Este ejercicio trata de capturar información desde el teclado (un nombre), para posteriormente almacenarla en una cadena de caracteres y mostrarla por pantalla en forma de saludo personalizado.

```
// fichero Saludo.java
// Este programa hace un saludo personalizado
import java.io.*;

public class Saludo {
    public static void main(String[] args) {
        try {
```

¹Al ser una *Preview Feature* en Java 21, el compilador requiere que se habilite explícitamente. Debes compilar usando el flag: `javac --release 21 --enable-preview Main.java` en Java 25 ya está incluida por defecto

2.2. Saludo personalizado

```
InputStreamReader leer = new InputStreamReader(System.in);
BufferedReader buff = new BufferedReader(leer);

System.out.println("¡Hola! ¿Cómo te llamas?");
String nombre = buff.readLine();

System.out.println("Qué tal estás, " + nombre + "?");
} catch (IOException ioex) {
    System.err.println("Error al leer la entrada: " +
        ioex.getMessage());
}
}
```

El ejercicio lo hemos resuelto utilizando la forma canónica y más bajo nivel de java, pero en la actualidad existe ya una librería que nos permite abstraernos de lo farragoso que es leer desde teclado usando la forma tradicional, la cual vamos a ver un ejemplo a continuación.

```
// fichero Saludo2.java
// Este programa hace un saludo personalizado
import java.util.Scanner;

public class Saludo {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        System.out.println("¡Hola! ¿Cómo te llamas?");
        String nombre = sc.nextLine();
        System.out.println("Qué tal estás " + nombre);
        sc.close();

    }
}
```

Nota: Con el uso de la clase Scanner, dependiendo de la configuración regional de tu sistema, deberás introducir los decimales con punto (.) o con coma (,) cuando trabajes con números double o float.

2.3. Capturar distinto tipo de información desde teclado y mostrarla por pantalla

Este ejercicio trata de capturar distinto tipo de información desde el teclado, para posteriormente almacenarla en su variable correspondiente, y mostrar dicha información por pantalla.

```
// fichero Capturar.java
// Este programa hace preguntas y con ello hace respuestas
import java.util.Scanner;

public class Capturar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.println("Dinos tu edad, peso y color favorito:");
            System.out.print("Edad: ");
            int edad = sc.nextInt();
            System.out.print("Peso (en Kg): ");
            float peso = sc.nextFloat();
            sc.nextLine();
            System.out.print("Color favorito: ");
            String color = sc.nextLine();
            System.out.println("¡El " + color + "!!!");
            System.out.printf("¿Cómo puede gustarte el %s con %d años\ny pesando %.2f Kg.?\n",
                              color, edad, peso);
        }
        catch (Exception e) {
            System.err.println("Entrada no válida. Asegúrate de\nintroducir los datos correctamente.");
        }
        finally {
            sc.close();
        }
    }
}
```

2.4. Calcular el área de cuadrado a partir de un lado

2.4. Calcular el área de cuadrado a partir de un lado

Este ejercicio calcula el área de un cuadrado a partir de un lado dado por teclado y muestra el resultado por pantalla. La fórmula del área de un cuadrado [4] viene dada por $A = l^2$.

```
// fichero AreaCuadrado.java
// Este programa calcula el área de un cuadrado con de uno de sus lados
import java.util.Scanner;

public class AreaCuadrado {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Introduce el valor del lado de un
                             cuadrado: ");
            double lado = sc.nextDouble();
            if (lado <= 0) {
                System.err.println("Error: el lado debe ser un número
                                   positivo.");
                return;
            }
            double area = lado * lado;
            System.out.printf("El área del cuadrado es: %.2f\n", area);
        }
        catch (java.util.InputMismatchException e) {
            System.err.println("Error: Debes introducir un valor
                               numérico válido.");
        }
        catch (Exception e) {
            System.err.println("Ocurrió un error inesperado.");
        }
        finally {
            sc.close();
        }
    }
}
```

2.5. Conversor de pesetas a Euros

Este ejercicio nos propone crear un conversor de pesetas a euros. Partiendo de un número dado por teclado, calcularemos su equivalencia a euros y mostraremos su cantidad expresada en euros por pantalla.

El cambio de euros a pesetas está fijado a razón de 1€= 166,386 pesetas.

```
// fichero PesetasEuros.java
import java.util.InputMismatchException;
import java.util.Scanner;

public class PesetasEuros {

    private static final double TASA_CONVERSION = 166.386;
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Introduzca su valor en pesetas: ");
            double pesetas = sc.nextDouble();
            double euros = pesetas / TASA_CONVERSION;
            System.out.printf("Su valor en euros es: %.2f%n", euros);
        }
        catch (InputMismatchException e) {
            System.err.println("Error de entrada: Debe introducir un
                               valor numérico válido.");
        }
        catch (Exception e) {
            System.err.println("Ocurrió un error inesperado durante la
                               conversión.");
        }
        finally {
            sc.close();
        }
    }
}
```

2.6. Suma, Resta, Multiplicación y División de dos números

Este ejercicio nos propone, dados dos números por teclado, realizar todas las operaciones aritméticas básicas sobre ellos y mostrar los resultados por pantalla.

```
// fichero OperacionesAritmeticas.java
import java.util.InputMismatchException;
import java.util.Scanner;
public class OperacionesAritmeticas {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double valor1 = 0; double valor2 = 0;
        boolean entradaValida = false;
        try {
            System.out.print("Introduzca el valor primero: ");
            valor1 = sc.nextDouble();
            System.out.print("Introduzca el valor segundo: ");
            valor2 = sc.nextDouble();
            entradaValida = true;
        }
        catch (InputMismatchException e) {
            System.err.println("Entrada inválida:deben ser valores numéricos");
        }
        if (entradaValida) {
            System.out.printf("Suma: %.2f\n", (valor1 + valor2));
            System.out.printf("Resta: %.2f\n", (valor1 - valor2));
            System.out.printf("Multiplicación: %.2f\n", (valor1 * valor2));
            if (valor2 == 0) {
                System.out.println("No es posible dividir por 0");
            } else {
                System.out.printf("División: %.2f\n", (valor1 /
                    valor2));
            }
        }
        sc.close();
    }
}
```

2.7. Precio medio de un producto en función de los precios de éste en otros establecimientos

Este ejercicio nos propone, dado por teclado los precios de un producto en 3 establecimientos, calcular el precio medio del producto y mostrarlo por pantalla.

```
// fichero PrecioMedio.java
// Este programa calcula el precio medio dados 3 precios
import java.util.Scanner;
import java.util.InputMismatchException;
public class PrecioMedio {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double precio1 = 0, precio2 = 0, precio3 = 0, media;
        boolean entradaValida = false;
        try {
            System.out.print("Introduzca el precio en establecimiento 1: ");
            precio1 = sc.nextDouble();
            System.out.print("Introduzca el precio en establecimiento 2: ");
            precio2 = sc.nextDouble();
            System.out.print("Introduzca el precio en establecimiento 3: ");
            precio3 = sc.nextDouble();
            entradaValida = true;
        } catch (InputMismatchException e) {
            System.err.println("Error: deben ser valores numéricos");
        }
        if (entradaValida) {
            if (precio1 < 0 || precio2 < 0 || precio3 < 0) {
                System.err.println("Error: Los precios no pueden ser <0");
            } else {
                media = (precio1 + precio2 + precio3) / 3;
                System.out.printf("El precio medio del producto es %.2f
euros%n", media);
            }
        }
        sc.close();
    }
}
```

2.8. Suma de los “n” primeros números naturales consecutivos

Este ejercicio nos propone, dado un número “n” por teclado, calcular los n primeros números naturales consecutivos y mostrar el resultado por pantalla.

Para ello utilizaremos la fórmula de las Series aritméticas Notables de números naturales [4], conocida como la fórmula de Gauss:

$$\sum_{i=1}^n i = 1 + 2 + 3 + \dots + n = \frac{n * (n + 1)}{2}$$

```
// fichero SumaNumerosNaturales.java
// Este programa calcula la suma de los "n" primeros números naturales
import java.util.InputMismatchException;
import java.util.Scanner;
public class SumaNumerosNaturales {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long n = 0; boolean entradaValida = false;
        try {
            System.out.print("Introduzca número de términos (n) a sumar: ");
            n = sc.nextLong();
            entradaValida = true;
        } catch (InputMismatchException e) {
            System.err.println("Error: Debe ser un número entero >0");
        }
        if (entradaValida) {
            if (n <= 0) {
                System.err.println("Error: El número de términos (n)
                debe ser mayor que cero.");
            } else {
                long suma = n * (n + 1) / 2;
                System.out.println("La suma de los " + n + " primeros
                números naturales es: " + suma);
            }
        }
        sc.close();
    }
}
```

2.9. Pasar de pulgadas a milímetros

Este ejercicio nos propone que, dado un valor numérico expresado en pulgadas por teclado, este sea pasado a milímetros y el resultado mostrado por pantalla.

Para ello se tendrá en cuenta que 1 pulgada = 25.4 mm.

```
// fichero PulgadasMilimetros.java
// Este programa cambia pulgadas por milímetros
import java.util.Scanner;

public class PulgadasMilimetros {
    private static final double FACTOR_CONVERSION = 25.4;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Introduzca valor en pulgadas: ");
            double pulgadas = sc.nextDouble();
            if (pulgadas < 0) {
                System.err.println("Error: el valor no puede ser negativo.");
            } else {
                double milimetros = FACTOR_CONVERSION * pulgadas;
                System.out.printf("El resultado en milímetros es: %.2f%n", milimetros);
            }
        }
        catch (Exception e) {
            System.err.println("Error: Debe introducir un número válido.");
        }
        finally {
            sc.close();
        }
    }
}
```

2.10. Pasar de grados Celsius a grados Fahrenheit

Este ejercicio nos propone que, dado por teclado un valor numérico expresado en grados Celsius (°C) (históricamente conocidos como centígrados), éste sea pasado a grados Fahrenheit (°F) y mostrado por pantalla. Para ello se tendrá en cuenta que:

$$F = \frac{9}{5}C + 32$$

```
// fichero CambioGrados.java
// Este programa cambia grados centígrados por fahrenheit
import java.util.Scanner;
import java.util.InputMismatchException;
public class CambioGrados {
    private static final double FACTOR_MULTIPLICACION = 9.0 / 5.0;
    private static final double SUMA_FINAL = 32.0;
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double centigrados = 0;
        boolean entradaValida = false;
        try {
            System.out.print("Introducir valor en grados Centígrados:
                ");
            centigrados = sc.nextDouble();
            entradaValida = true;
        } catch (InputMismatchException e) {
            System.err.println("Error de entrada: Debe introducir un
                valor numérico válido.");
        }
        if (entradaValida) {
            // Cálculo usando la fórmula: F = (C * 9/5) + 32
            double fahrenheit = (centigrados * FACTOR_MULTIPLICACION)
                + SUMA_FINAL;
            System.out.printf("El resultado en grados Fahrenheit es:
                %.1fÂ°F%n", fahrenheit);
        }
        sc.close();
    }
}
```

2.11. Calcular estadísticas de suspensos, aprobados, notables y sobresalientes de la clase

Este ejercicio nos propone, dados por teclado el número de suspensos, aprobados, notables y sobresalientes de una clase calcular:

- El porcentaje de alumnos que superan la asignatura
- El porcentaje de alumnos que suspenden la asignatura
- El porcentaje de alumnos que sacan notable en la asignatura
- El porcentaje de alumnos que sacan sobresaliente en la asignatura
- El porcentaje de alumnos que han sacado un aprobado en la asignatura

```
// fichero Estadisticas.java
// Este programa calcula estadísticas sobre alumnos
import java.util.Scanner;
import java.util.InputMismatchException;
public class Estadisticas {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Introduce total de suspensos: ");
            long suspensos = sc.nextLong();
            System.out.print("Introduce total de aprobados: ");
            long aprobados = sc.nextLong();
            System.out.print("Introduce total de notables: ");
            long notables = sc.nextLong();
            System.out.print("Introduce total de sobresalientes: ");
            long sobresalientes = sc.nextLong();
            long totalAlumnos = suspensos + aprobados + notables +
                sobresalientes;
            if (totalAlumnos == 0) {
                System.err.println("Error: El total de alumnos no
                    puede ser cero.");
                return;
            }
        }
    }
}
```

2.11. Calcular estadísticas de suspensos, aprobados, notables y sobresalientes de la clase

```
        if (suspensos < 0 || aprobados < 0 || notables < 0 ||
            sobresalientes < 0) {
            System.err.println("Error: Los valores no pueden ser
                                negativos.");
            return;
        }
        double porcentajeSuspensos = (suspensos / (double)
            totalAlumnos) * 100;
        double porcentajeAprobados = (aprobados / (double)
            totalAlumnos) * 100;
        double porcentajeNotables = (notables / (double)
            totalAlumnos) * 100;
        double porcentajeSobresalientes = (sobresalientes /
            (double) totalAlumnos) * 100;
        double porcentajeSuperan = ((aprobados + notables +
            sobresalientes) / (double) totalAlumnos) * 100;
        System.out.printf("Total de alumnos: %d\n", totalAlumnos);
        System.out.printf("Porcentaje que superan la asignatura:
                            %.2f%%\n", porcentajeSuperan);
        System.out.printf("Porcentaje de suspensos: %.2f%%\n",
            porcentajeSuspensos);
        System.out.printf("Porcentaje de aprobados: %.2f%%\n",
            porcentajeAprobados);
        System.out.printf("Porcentaje de notables: %.2f%%\n",
            porcentajeNotables);
        System.out.printf("Porcentaje de sobresalientes:
                            %.2f%%\n", porcentajeSobresalientes);
    } catch (InputMismatchException e) {
        System.err.println("Error: Introduce solo números enteros
                            válidos.");
    } finally {
        sc.close();
    }
}
```

2.12. Cálculo del precio de venta de un coche a partir del coste de fabricación, ganancia de la empresa e IVA

Este ejercicio nos propone, dado por teclado un valor numérico que expresa el coste de fabricar un coche, calcular el precio total de venta al que tendrían que venderlo, si queremos una ganancia fija de un 15 % y el valor del IVA del coche es de 21%.

```
// fichero CosteCoche.java
// Este programa calcula el precio total de un coche
import java.util.Scanner;
import java.util.InputMismatchException;
public class CosteCoche {
    static final double GANANCIA_EMPRESA = 1.15, IMPUESTO_IVA = 1.21;
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double costeCoche, pvAntesImpuestos, precioTotal;
        System.out.println("Introduce el coste del vehículo: ");
        try {
            costeCoche = scanner.nextDouble();
            if (costeCoche <= 0) {
                System.err.println("Error: El coste del vehículo debe ser >0");
                return;
            }
            pvAntesImpuestos = costeCoche * GANANCIA_EMPRESA;
            precioTotal = pvAntesImpuestos * IMPUESTO_IVA;
            System.out.printf("El precio total del automóvil es: %.2f%n",
                precioTotal);
        } catch (InputMismatchException e) {
            System.err.println("Error: Por favor, introduce un valor
                numérico válido.");
        } catch (Exception e) {
            System.err.println("Ha ocurrido un error inesperado.");
            e.printStackTrace();
        } finally {
            scanner.close();
        }
    }
}
```

2.13. Calcular el perímetro de una circunferencia a partir del radio

Este ejercicio nos propone, dado por teclado el radio de una circunferencia, calcular el perímetro de la misma y mostrarlo por pantalla. Para ello, se tendrá en cuenta que el perímetro de una circunferencia [4] viene dado por la siguiente fórmula $P = 2\pi r$.

```
// fichero PerimetroCir.java
// Este programa calcula el perímetro de una circunferencia
import java.util.Scanner;
import java.util.InputMismatchException;

public class PerimetroCir {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.print("Introduzca el radio: ");
            double radio = scanner.nextDouble();
            if (radio <= 0) {
                System.err.println("Error: El radio debe ser un número
                    positivo");
                return;
            }
            double perimetro = 2 * Math.PI * radio;
            System.out.printf("El perímetro de la circunferencia es:
                %.4f%n", perimetro);
        } catch (InputMismatchException e) {
            System.err.println("Error: Por favor, introduce un valor
                numérico válido");
        } catch (Exception e) {
            System.err.println("Ha ocurrido un error inesperado.");
            e.printStackTrace();
        } finally {
            scanner.close();
        }
    }
}
```

2.14. Calcular la raíz cuadrada de un número dado

Este ejercicio nos propone, dado por teclado un número, calcular su raíz cuadrada y mostrarla por pantalla. Para ello utilizaremos la función `sqrt` que proporciona la librería `Math` de Java.

```
// fichero RaizCuadrada.java
// Este programa calcula la raíz cuadrada de un número
import java.util.Scanner;
import java.util.InputMismatchException;

public class RaizCuadrada {

    public static void main(String[] args) {
        try (Scanner scanner = new Scanner(System.in)) {
            System.out.print("Introduce el número a calcular la raíz
                             cuadrada: ");
            double numero = scanner.nextDouble();
            if (numero < 0) {
                System.err.println("Error: No se puede calcular la raíz
                                   cuadrada real de un número negativo.");
                return;
            }
            double resultado = Math.sqrt(numero);
            System.out.printf("La raíz cuadrada de %.2f es: %.4f\n", numero,
                             resultado);
        } catch (InputMismatchException e) {
            System.err.println("Error: Por favor, introduce un valor
                               numérico válido.");
        } catch (Exception e) {
            System.err.println("Ha ocurrido un error inesperado.");
            e.printStackTrace();
        }
    }
}
```

2.15. Calcular la raíz n-ésima de un número

Este ejercicio nos propone, dado por teclado un número entero y un exponente, calcular la raíz n-ésima de dicho número y mostrarla por pantalla. Para ello, debemos tener en cuenta que la raíz n-ésima de un número $\sqrt[n]{a}$ se puede reescribir como $a^{\frac{1}{n}}$ [4].

```
// fichero RaizEnesima.java
// Este programa calcula la raíz n-ésima de un número
import java.util.Scanner;
import java.util.InputMismatchException;
public class RaizEnesima {
    public static void main(String[] args) {
        try (Scanner scanner = new Scanner(System.in)) {
            System.out.print("Introduce el número al que deseas calcular la
                raíz: ");
            double numero = scanner.nextDouble();
            System.out.print("Introduce el exponente de la raíz (n): ");
            double exponente = scanner.nextDouble();
            if (exponente == 0) {
                System.err.println("Error: El exponente no puede ser cero.");
                return;
            }
            if (numero < 0 && exponente % 2 == 0) {
                System.err.println("No existe raíz par real de un número <0");
                return;
            }
            double resultado = Math.pow(numero, 1.0 / exponente);
            System.out.printf("La raíz %.0f-ésima de %.4f es: %.6f\n",
                exponente, numero, resultado);
        } catch (InputMismatchException e) {
            System.err.println("Error: Por favor, introduce valores
                numéricos válidos.");
        } catch (Exception e) {
            System.err.println("Ha ocurrido un error inesperado.");
            e.printStackTrace();
        }
    }
}
```

2.16. Calcular el área de un triángulo rectángulo

Este ejercicio nos propone, dados la base de un triángulo y la altura por teclado, calcular el área del mismo y mostrarla por pantalla. Para ello, utilizaremos la fórmula del área de un triángulo rectángulo [19] $A = \frac{b \cdot a}{2}$

```
// fichero AreaTriangulo1.java
// Este programa calcula el área de un triángulo rectángulo a partir
// de la base y la altura
import java.util.Scanner;
import java.util.InputMismatchException;

public class AreaTriangulo1 {
    public static void main(String[] args) {
        try (Scanner scanner = new Scanner(System.in)) {
            System.out.print("Introduce la base del triángulo rectángulo: ");
            double base = scanner.nextDouble();
            System.out.print("Introduce la altura del triángulo rectángulo: ");
            double altura = scanner.nextDouble();
            if (base <= 0 || altura <= 0) {
                System.err.println("Error: La base y la altura deben ser valores positivos.");
                return;
            }
            double area = (base * altura) / 2.0;
            System.out.printf("El área del triángulo rectángulo es: %.4f%n", area);
        } catch (InputMismatchException e) {
            System.err.println("Error: Por favor, introduce valores numéricos válidos.");
        } catch (Exception e) {
            System.err.println("Ha ocurrido un error inesperado.");
            e.printStackTrace();
        }
    }
}
```

2.17. Calcular el área de un triángulo equilátero

Este ejercicio nos propone, dado un lado de un triángulo equilátero por teclado, calcular el área del mismo y mostrarla por pantalla. Para ello, utilizaremos la fórmula del área de un triángulo equilátero [19] de $A = \frac{\sqrt{3}}{4} \cdot a^2$

```
// fichero AreaTriangulo2.java
// Este programa calcula el área de un triángulo equilátero a partir
// de uno de sus lados
import java.util.Scanner;
import java.util.InputMismatchException;

public class AreaTriangulo2 {

    public static void main(String[] args) {

        try (Scanner scanner = new Scanner(System.in)) {

            System.out.print("Introduce un lado del triángulo equilátero: ");
            double lado = scanner.nextDouble();
            if (lado <= 0) {
                System.err.println("Error: El lado del triángulo debe ser un
                    número positivo.");
                return;
            }
            double area = (Math.sqrt(3) / 4) * lado * lado;
            System.out.printf("El área del triángulo equilátero de lado %.4f
                es: %.6f\n", lado, area);
        } catch (InputMismatchException e) {
            System.err.println("Error: Por favor, introduce un valor
                numérico válido para el lado.");
        } catch (Exception e) {
            System.err.println("Ha ocurrido un error inesperado.");
            e.printStackTrace();
        }
    }
}
```

2.18. Calcular el área de cualquier triángulo a partir de sus lados mediante la fórmula de Herón

Este ejercicio nos propone, dados por teclado los valores de los lados de un triángulo, calcular su área y mostrarla por pantalla utilizando la fórmula de Herón.

El área de un triángulo según la fórmula de Herón [19] viene dada por:

$$A = \sqrt{sp * (sp - l1) * (sp - l2) * (sp - l3)}$$

donde

$$sp = \frac{(l1 + l2 + l3)}{2}$$

```
// fichero AreaTriangulo.java
// Este programa calcula el área de un triángulo a partir de sus lados
// mediante la fórmula de Herón
import java.util.Scanner;
import java.util.InputMismatchException;

public class AreaTriangulo {

    public static void main(String[] args) {

        try (Scanner scanner = new Scanner(System.in)) {

            System.out.print("Introduce la longitud del primer lado: ");
            double l1 = scanner.nextDouble();

            System.out.print("Introduce la longitud del segundo lado: ");
            double l2 = scanner.nextDouble();

            System.out.print("Introduce la longitud del tercer lado: ");
            double l3 = scanner.nextDouble();

            if (l1 <= 0 || l2 <= 0 || l3 <= 0) {
                System.err.println("Error: Los lados del triángulo deben ser
                    positivos.");
                return;
            }
        }
    }
}
```

2.19. Calcular el volumen de un cilindro a partir de su altura y diámetro

```
}

if (l1 + l2 <= l3 || l1 + l3 <= l2 || l2 + l3 <= l1) {
    System.err.println("Error: Los lados introducidos no forman un
        triángulo válido.");
    return;
}

double sp = (l1 + l2 + l3) / 2.0;
double area = Math.sqrt(sp * (sp - l1) * (sp - l2) * (sp - l3));
System.out.printf("El área del triángulo es: %.4f%n", area);

} catch (InputMismatchException e) {
    System.err.println("Error: Por favor, introduce valores
        numéricos válidos.");
} catch (Exception e) {
    System.err.println("Ha ocurrido un error inesperado.");
    e.printStackTrace();
}
}
```

2.19. Calcular el volumen de un cilindro a partir de su altura y diámetro

Este ejercicio nos propone, dados por teclado el diámetro y la altura de un cilindro, calcular el volumen del mismo y mostrarlo por pantalla.

Para ello se tendrá en cuenta que $V = \pi * r^2 * H$ [4], siendo r el radio de la circunferencia y H la altura del cilindro. El radio se obtiene a partir del diámetro tal que $r = d/2$.

```
// fichero VolumenCilindro.java
// Este programa calcula el volumen de un cilindro
import java.util.Scanner;
import java.util.InputMismatchException;

public class VolumenCilindro {

    public static void main(String[] args) {

        try (Scanner scanner = new Scanner(System.in)) {

            System.out.print("Introduzca el diámetro, en metros: ");
            double diametro = scanner.nextDouble();
            System.out.print("Introduzca la altura, en metros: ");
            double altura = scanner.nextDouble();

            if (diametro <= 0 || altura <= 0) {
                System.err.println("Error: El diámetro y la altura deben ser
                    valores positivos.");
                return;
            }

            double radio = diametro / 2.0;
            double volumen = Math.PI * Math.pow(radio, 2) * altura;
            System.out.printf("El volumen del cilindro es de %.4f metros
                cúbicos%n", volumen);

        } catch (InputMismatchException e) {
            System.err.println("Error: Por favor, introduzca valores
                numéricos válidos.");
        } catch (Exception e) {
            System.err.println("Ha ocurrido un error inesperado.");
            e.printStackTrace();
        }
    }
}
```

2.20. Calcular el valor del lado “a” de un triángulo rectángulo dados el valor de la hipotenusa “h” y del lado “b”

2.20. Calcular el valor del lado “a” de un triángulo rectángulo dados el valor de la hipotenusa “h” y del lado “b”

Este ejercicio nos propone, dados por teclado el valor del lado “b” de un triángulo rectángulo y el valor de su hipotenusa “h”, calcular el valor del lado “a” y mostrarlo por pantalla. Para ello, se utilizará el Teorema de Pitágoras $h^2 = a^2 + b^2$ [19] que despejando nos queda que $a = \sqrt{h^2 - b^2}$

```
// fichero LadoTriangulo.java
// Este programa calcula el valor del lado a de un triangulo
// rectángulo usando el valor del lado b y la hipotenusa h
import java.util.Scanner;
import java.util.InputMismatchException;
public class LadoTriangulo {
    public static void main(String[] args) {
        try (Scanner scanner = new Scanner(System.in)) {
            System.out.print("Introduzca el valor del lado 'b': ");
            double b = scanner.nextDouble();
            System.out.print("Introduzca el valor de la hipotenusa 'h': ");
            double h = scanner.nextDouble();
            if (b <= 0 || h <= 0) {
                System.err.println("Error: Los lados deben ser positivas.");
                return;
            }
            if (b >= h) {
                System.err.println("Error: La hipotenusa debe ser >b.");
                return;
            }
            double a = Math.sqrt(Math.pow(h, 2) - Math.pow(b, 2));
            System.out.printf("El valor del lado 'a' es: %.4f\n", a);
        } catch (InputMismatchException e) {
            System.err.println("Introduzca valores numéricos válidos");
        } catch (Exception e) {
            System.err.println("Ha ocurrido un error inesperado.");
            e.printStackTrace();
        }
    }
}
```

2.21. Calcular la altura de un triángulo equilátero

Este ejercicio nos propone, dado por teclado el lado de un triángulo equilátero, calcular la altura h del mismo y mostrar el resultado por pantalla. Para calcular la altura de un triángulo equilátero, aplicaremos el Teorema de Pitágoras obteniendo que $h = \frac{\sqrt{3}a}{2}$ [19].

```
// fichero AlturaTriangulo.java
// Este programa calcula la altura de un triángulo equilátero
import java.util.Scanner;
import java.util.InputMismatchException;
public class AlturaTriangulo {
    public static void main(String[] args) {
        try (Scanner scanner = new Scanner(System.in)) {

            System.out.print("Introduzca el lado de un triángulo equilátero:
                ");
            double lado = scanner.nextDouble();

            if (lado <= 0) {
                System.err.println("Error: El lado debe ser un valor
                    positivo.");
                return;
            }
            double altura = (Math.sqrt(3) * lado) / 2.0;
            System.out.printf("La altura de un triángulo equilátero de lado
                %.4f es: %.4f%n", lado, altura);

        } catch (InputMismatchException e) {
            System.err.println("Error: Por favor, introduzca un valor
                numérico válido.");
        } catch (Exception e) {
            System.err.println("Ha ocurrido un error inesperado.");
            e.printStackTrace();
        }
    }
}
```

2.22. Obtener el valor del binomio de suma al cuadrado $(a + b)^2$

2.22. Obtener el valor del binomio de suma al cuadrado $(a + b)^2$

Este ejercicio nos propone, dados por teclado dos números, calcular el binomio de suma al cuadrado $(a + b)^2$ [4] y mostrar el resultado por pantalla

```
// fichero BinomioSuma.java
// Este programa calcula el binomio de suma (a+b)^2

import java.util.Scanner;
import java.util.InputMismatchException;

public class BinomioSuma {

    public static void main(String[] args) {

        try (Scanner scanner = new Scanner(System.in)) {

            System.out.print("Introduce el valor de a: ");
            double a = scanner.nextDouble();

            System.out.print("Introduce el valor de b: ");
            double b = scanner.nextDouble();

            double resultado = Math.pow(a, 2) + 2 * a * b + Math.pow(b, 2);

            System.out.printf("El resultado de (a+b)^2 es: %.4f%n",
                             resultado);

        } catch (InputMismatchException e) {
            System.err.println("Error: Por favor, introduce valores
                               numéricos válidos.");
        } catch (Exception e) {
            System.err.println("Ha ocurrido un error inesperado.");
            e.printStackTrace();
        }
    }
}
```


Capítulo 3

Flujos de control, funciones, cadenas de caracteres, arrays y matrices

Este capítulo presenta una serie de problemas resueltos que ilustran distintos tipos de flujos de control. También se incluyen ejercicios sobre cómo resolver problemas utilizando procedimientos o funciones, así como su integración en el programa principal. Por último, se abordarán problemas que implican el trabajo con cadenas de caracteres, arrays y matrices. Para el trabajo con caracteres, arrays y matrices no utilizaremos ninguna librería específica ni memoria dinámica. Trabajaremos con arrays nativos de tamaño fijo, sin utilizar colecciones dinámicas auto-redimensionables (como ArrayList). Para revisar la teoría y especificaciones propias del lenguaje para abordar flujos de control, funciones, cadenas de caracteres, arrays y matrices, proponemos al lector revisar algunos de los libros más destacados en la bibliografía tales como [16] [6] [14].

A continuación, presentamos los ejercicios resueltos para este capítulo.

3.1. ¿Número par o impar?

Este ejercicio nos propone, dado un número entero, calcular si éste es par o impar y mostrarlo por pantalla. Para ello utilizaremos el flujo de control IF - ELSE.

```
// fichero ParImpar.java
// Este programa indica si un número entero es par o impar
import java.util.Scanner;
import java.util.InputMismatchException;

public class ParImpar {

    public static void main(String[] args) {

        try (Scanner scanner = new Scanner(System.in)) {

            System.out.print("Introduzca un número entero: ");
            int numero = scanner.nextInt();
            if (numero % 2 == 0) {
                System.out.println("El número " + numero + " es PAR");
            } else {
                System.out.println("El número " + numero + " es IMPAR");
            }
        } catch (InputMismatchException e) {
            System.err.println("Error: Por favor, introduzca un número entero válido.");
        } catch (Exception e) {
            System.err.println("Ha ocurrido un error inesperado.");
            e.printStackTrace();
        }
    }
}
```

3.2. Calcular si un número entero es múltiplo de 3 y a la vez par

Este ejercicio nos propone, dado un número entero, calcular si es múltiplo de 3 y a la vez par y mostrarlo por pantalla.

```
// fichero Multiplo3.java
// Este programa dice si un número es múltiplo de 3 y par
import java.util.Scanner;

public class Multiplo3 {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Introduzca un número: ");
            long numero = sc.nextLong();
            if (numero % 3 == 0 && numero % 2 == 0) {
                System.out.println("El número " + numero + " es
                                múltiplo de 3 y también es par (Múltiplo de 6).");
            } else if (numero % 3 == 0) {
                System.out.println("El número " + numero + " es
                                múltiplo de 3.");
            } else {
                System.out.println("El número " + numero + " NO es
                                múltiplo de 3.");
            }
        } catch (java.util.InputMismatchException e) {
            System.out.println("Error: debe introducir un número
                                entero.");
        } finally {
            sc.close();
        }
    }
}
```

3.3. Calcular si un año es bisiesto

Este ejercicio nos propone, dado un año (entero positivo), calcular si es bisiesto o no y mostrarlo por pantalla.

Nota: Un año es bisiesto en el calendario Gregoriano, si es divisible entre 4, excepto aquellos divisibles entre 100 pero no entre 400.

```
// fichero Bisiesto.java
// Este programa dice si un año es bisiesto o no
import java.util.Scanner;

public class Bisiesto {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Introduzca un año: ");
            int anio = sc.nextInt();
            if ((anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0))
                System.out.println("El año " + anio + " ES BISIESTO");
            else
                System.out.println("El año " + anio + " NO ES BISIESTO");
        } catch (java.util.InputMismatchException e) {
            System.out.println("Error: debe introducir un número entero (por ejemplo, 2024).");
        } finally {
            sc.close();
        }
    }
}
```

3.4. Programa que calcule el Máximo Común Divisor de dos números enteros no negativos mediante el algoritmo de Euclides

3.4. Programa que calcule el Máximo Común Divisor de dos números enteros no negativos mediante el algoritmo de Euclides

Este ejercicio nos propone, dado dos números enteros no negativos, calcular el máximo común divisor de dichos números utilizando el algoritmo de Euclides (más eficiente) y mostrarlo por pantalla.

El **Algoritmo de Euclides** es un método eficiente para encontrar el Máximo Común Divisor (MCD) de dos números enteros no negativos a y b [9]. Se basa en el principio de que el MCD de dos números también divide a su diferencia, lo que permite reducir el problema a números más pequeños. El algoritmo se basa en la siguiente propiedad [4] [9]: Si a y b son dos números enteros positivos con $a > b$, entonces $\text{MCD}(a, b) = \text{MCD}(b, r)$, donde r es el resto de la división euclídea de a entre b . Es decir: $\text{MCD}(a, b) = \text{MCD}(b, a \bmod b)$

```
// fichero Mcd.java
// Este programa calcula el MCD dados 2 números
import java.util.Scanner;
public class Mcd {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Introduzca el valor de a: ");
            long a = sc.nextLong();
            System.out.print("Introduzca el valor de b: ");
            long b = sc.nextLong();
            long a2 = a; long b2 = b;
            while (a != 0) {
                long temporal = a;
                a = b % a;
                b = temporal;
            }
            System.out.println("El M.C.D. de " + a2 + " y " + b2 + " es: " + b);
        } catch (java.util.InputMismatchException e) {
            System.out.println("Debe introducir números enteros válidos");
        } finally {sc.close();}
    }
}
```

3.5. Función para calcular del Máximo Común Divisor de dos números enteros no negativos mediante el algoritmo de Euclides

Este ejercicio nos propone crear una función para, dados dos números enteros no negativos, calcular el máximo común divisor de dichos números utilizando el algoritmo de Euclides (visto en el ejercicio anterior) y que devuelva el resultado. Para ello, convertiremos el ejercicio anterior 3.4 a una función y mostraremos el resultado.

```
// fichero Mcd1.java
// Este programa calcula el MCD dados 2 números mediante una función
import java.util.Scanner;
import java.util.InputMismatchException;
public class Mcd1 {
    public static long MCD(long a, long b) {
        long temporal;
        a = Math.abs(a); b = Math.abs(b);
        while (a != 0) {
            temporal = a;
            a = b % a;
            b = temporal;
        } return b;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Introduzca valor de a: ");
            long a = sc.nextLong();
            System.out.print("Introduzca valor de b: ");
            long b = sc.nextLong();
            long resu = MCD(a, b);
            System.out.println("EL M.C.D de " + a + " y " + b + " es: " + resu);
        } catch (InputMismatchException e) {
            System.out.println("Debe introducir números enteros válidos.");
        } finally {sc.close();}
    }
}
```

3.6. Función para calcular si un número entero es divisor de otro

Este ejercicio nos propone realizar una función que calcule si *a* es divisor de *b*. Un número entero *d* es divisor de otro número entero *N* si, al dividir *N* por *d*, el resto de la división es cero [4]. Para ello, utilizaremos la función módulo % de C, la cual nos devuelve el resto de la división entre 2 números.

```
// fichero Divisor.java
// Este programa calcula si un número a es divisor de otro número b
import java.util.Scanner;
public class Divisor {
    public static boolean divisible(long a, long b) {
        return b != 0 && b % a == 0;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Introduzca valor de a: ");
            long a = sc.nextLong();
            System.out.print("Introduzca valor de b: ");
            long b = sc.nextLong();
            if (divisible(a, b)) {
                System.out.println(a + " es divisible entre " + b);
            } else if (b == 0) {
                System.out.println("Error: no se puede dividir entre
                    0.");
            } else {
                System.out.println(a + " NO es divisible entre " + b);
            }
        } catch (java.util.InputMismatchException e) {
            System.out.println("Error: debe introducir números
                enteros.");
        } finally {
            sc.close();
        }
    }
}
```

3.7. Pasar a función el ejercicio propuesto número 2.18

Este ejercicio toma como base el ejercicio 2.18 para crear una función que devuelva el valor del área de un triángulo usando la fórmula de Herón.

```
// fichero AreaTriangulo2.java
// Este programa calcula el área de un triángulo a partir de sus
// lados mediante la fórmula de Herón

import java.util.Scanner;
import java.util.InputMismatchException;

public class AreaTriangulo2 {

    public static double AreaTrianguloHeron(double l1, double l2,
        double l3) {
        double sp = (l1 + l2 + l3) / 2;
        return Math.sqrt((sp * (sp - l1) * (sp - l2) * (sp - l3)));
    }

    public static boolean esTrianguloValido(double a, double b, double
        c) {
        return (a + b > c) && (a + c > b) && (b + c > a);
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        try {
            double l1x, l2x, l3x;
            System.out.print("Introduce la longitud del primer lado: ");
            l1x = sc.nextDouble();
            System.out.print("Introduce la longitud del segundo lado: ");
            l2x = sc.nextDouble();
            System.out.print("Introduce la longitud del tercer lado: ");
            l3x = sc.nextDouble();
            if (esTrianguloValido(l1x, l2x, l3x)) {
                double area = AreaTrianguloHeron(l1x, l2x, l3x);
                System.out.printf("\nEl área del triángulo con lados %.2f,
```

3.8. Función para calcular el área de un circunferencia de radio válido

```
        %.2f y %.2f es: %.2f\n", l1x, l2x, l3x, area);
    } else {
        System.out.println("\nError: Las longitudes ingresadas NO
        pueden formar un triángulo (Fallo de la Desigualdad
        Triangular).");
    }

}

} catch (InputMismatchException e) {
    System.out.println("\nError de entrada: Debes introducir un
    valor numérico válido para las longitudes.");
} finally {
    sc.close();
}
}
```

3.8. Función para calcular el área de un circunferencia de radio válido

Este ejercicio nos propone calcular el área de una circunferencia. Al igual que en el anterior ejercicio, en primer lugar aparece la función que se ha desarrollado y seguidamente un ejemplo de su uso. Para este caso, se ha implementado la lectura perfecta de un número para que el radio sea correcto.

```
// fichero AreaCircunferencia.java
// Este programa calcula el área de una circunferencia dado un radio
// válido

import java.util.Scanner;
import java.util.InputMismatchException;

public class AreaCircunferencia {

    public static double AreaCircunferencia_(double r) {
        return Math.PI * Math.pow(r, 2);
    }
}
```

```
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);
    try {
        double radio;

        System.out.print("Introduce el radio de la circunferencia:
        ");
        radio = sc.nextDouble();

        while (radio <= 0) {
            System.out.println("ERROR: El radio debe ser mayor que
            cero.\n");
            System.out.print("Introduzca radio: ");
            radio = sc.nextDouble();
        }
        System.out.printf("El área de la circunferencia es:
        %.2f\n", AreaCircunferencia_(radio));
    } catch (InputMismatchException e) {
        System.out.println("Error: debes introducir un valor
        numérico válido para el radio.");
    } finally {
        sc.close();
    }
}
```

3.9. Calcular la suma de los “n” primeros números naturales (Con bucle)

Este ejercicio nos propone realizar la suma de los “n” primeros números naturales y mostrar el resultado de la suma por pantalla. Para ello, se ha utilizado un bucle **FOR** para practicar con los mismos. Como se puede observar, es mucho menos eficiente que el ejercicio planteado en 2.8.

```
// fichero SumaNumerosNaturales.java
// Este programa calcula la suma de los "n" primeros números naturales
import java.util.Scanner;
import java.util.InputMismatchException;

public class SumaNumerosNaturales {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Introduzca número de números naturales a
                sumar: ");
            long numero = sc.nextLong();
            if (numero <= 0) {
                System.out.println("Error: debe introducir un número
                    positivo.");
            } else {
                long suma = 0;
                for (long i = 1; i <= numero; i++) {
                    suma += i;
                }
                System.out.println("La suma de los " + numero + "
                    primeros números naturales es: " + suma);
            }
        } catch (InputMismatchException e) {
            System.out.println("Error: debe introducir un número
                entero válido.");
        } finally {sc.close();}
    }
}
```

3.10. Realizar un programa que sume números introducidos por teclado hasta que se escriba un número concreto de parada

Este ejercicio nos propone realizar la suma de números introducidos por teclado, hasta que se introduzca un número de parada (en nuestro caso el número -50) y muestre el resultado de la suma de dichos números por pantalla.

```
// fichero SumaNumeros.java
// Calcula la suma de números introducidos por teclado hasta que se
// introduce el número -50,
import java.util.Scanner;
import java.util.InputMismatchException;

public class SumaNumeros {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        double suma = 0;
        double numero = 0;

        System.out.println("Introduzca números para sumar. Escriba -50
            para ver el resultado.");
        do {
            try {
                System.out.print("Introduzca número: ");
                if (sc.hasNextDouble()) {
                    numero = sc.nextDouble();
                    if (numero != -50) {
                        suma += numero;
                    }
                } else {
                    String entradaErronea = sc.next();
                    System.out.println("Error: " + entradaErronea + "
                        no es un número. Inténtelo de nuevo.");
                    numero = 0;
                }
            }
        }
```

3.11. Dados 10 números por teclado decir cuál es el mayor de todos utilizando una función

```
        } catch (Exception e) {
            System.out.println("Ha ocurrido un error inesperado.
                               Terminando programa.");
            break;
        }
    } while (numero != -50);
    System.out.printf("La suma total de los números introducidos es:
                      %.2f\n", suma);
    sc.close();
}
}
```

3.11. Dados 10 números por teclado decir cuál es el mayor de todos utilizando una función

Este ejercicio nos propone, dados 10 números enteros introducidos por teclado, calcular cuál es el mayor de todos los introducidos. Para ello, vamos a intentar resolverlo guardando los datos introducidos en un array de enteros y posteriormente aplicando una función, la cual debe decir cuál es el número mayor introducido. Finalmente, mostraremos dicho número. Obviamente este ejercicio se puede resolver de una manera mucho más sencilla, pero se pretende practicar el uso de funciones y de arrays.

```
// fichero Mayor.java
// Este programa calcula cuál es el número mayor de 10 introducidos
// por teclado

import java.util.Scanner;
import java.util.InputMismatchException;

public class Mayor {

    public static long mayor(long[] numeros) {
        long max = numeros[0];
        for (int i = 1; i < numeros.length; i++) {
            if (numeros[i] > max) {
                max = numeros[i];
            }
        }
    }
}
```

```
    }  
    }  
    return max;  
}  
  
public static void main(String[] args) {  
  
    final int NUM_ELEMENTOS = 10;  
    Scanner sc = new Scanner(System.in);  
    long[] numerosin = new long[NUM_ELEMENTOS];  
  
    try {  
  
        for (int i = 0; i < NUM_ELEMENTOS; i++) {  
            System.out.printf("Introduzca número %d de %d: ", i +  
                1, NUM_ELEMENTOS);  
            numerosin[i] = sc.nextLong();  
        }  
  
        long maximo = mayor(numerosin);  
        System.out.println("\nEl número mayor de todos los  
            introducidos es: " + maximo);  
  
    } catch (InputMismatchException e) {  
        System.out.println("\nError de entrada: Debe introducir un  
            número entero válido.");  
    } catch (Exception e) {  
        System.out.println("\n Ha ocurrido un error inesperado: "  
            + e.getMessage());  
    } finally {  
        sc.close();  
    }  
}  
}
```

3.12. Calcular el tipo de triángulo en función de los lados

Este ejercicio nos propone, dados los lados de un triángulo, calcular el tipo de triángulo que es y mostrarlo por la pantalla.

```
// fichero TipoTriangulo.java
// Este programa calcula el tipo de triángulo en función de los lados

import java.util.Scanner;
import java.util.InputMismatchException;

public class TipoTriangulo {

    // Tolerancia para comparar doubles
    private static final double EPSILON = 1e-9;
    public static boolean esTrianguloValido(double a, double b, double
        c) {
        return (a > 0 && b > 0 && c > 0) &&
            (a + b > c) &&
            (a + c > b) &&
            (b + c > a);
    }
    // Función para comparar doubles con tolerancia
    public static boolean iguales(double x, double y) {
        return Math.abs(x - y) < EPSILON;
    }
    public static double leerLado(Scanner sc, String mensaje) {
        double lado = -1;
        while (true) {
            try {
                System.out.print(mensaje);
                lado = sc.nextDouble();
                if (lado > 0) {
                    break;
                } else {
                    System.out.println("Error: el valor debe ser mayor
                        que 0.");
                }
            }
        }
    }
}
```

```
        } catch (InputMismatchException e) {
            System.out.println("Error: debes introducir un número
                                válido.");
            sc.next();
        }
    }
    return lado;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    double l1, l2, l3;

    l1 = leerLado(sc, "Introduce la longitud del primer lado (l1):
    ");
    l2 = leerLado(sc, "Introduce la longitud del segundo lado
    (l2): ");
    l3 = leerLado(sc, "Introduce la longitud del tercer lado (l3):
    ");
    if (!esTrianguloValido(l1, l2, l3)) {
        System.out.println("\nError: los lados introducidos NO
        forman un triángulo válido.");
        return;
    }
    if (iguales(l1, l2) && iguales(l2, l3)) {
        System.out.println("El Triángulo es: EQUILÁTERO (Tres
        lados iguales)");
    } else if (iguales(l1, l2) || iguales(l1, l3) || iguales(l2,
    l3)) {
        System.out.println("El Triángulo es: ISÓSCELES (Dos lados
        iguales)");
    } else {
        System.out.println("El Triángulo es: ESCALENO (Todos los
        lados diferentes)");
    }
    sc.close();
}
}
```

3.13. Resolver ecuaciones de segundo grado mediante la fórmula general (sólo soluciones reales)

3.13. Resolver ecuaciones de segundo grado mediante la fórmula general (sólo soluciones reales)

Este ejercicio nos propone resolver ecuaciones de segundo grado, utilizando para ello la fórmula general $ax^2 + bx + c = 0$ [4], donde siempre $a \neq 0$. Para una ecuación de segundo grado con coeficientes reales, existen siempre dos soluciones, no necesariamente distintas, llamadas raíces.

A continuación, escribimos la fórmula general para obtener las raíces reales [4]:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

```
// fichero Ecuaciones.java
// Este programa calcula ecuaciones de segundo grados con railes reales
import java.util.InputMismatchException;
import java.util.Scanner;

public class Ecuaciones {
    // Procedimiento que calcula y muestra las raíces reales de la
    // ecuación
    static void resolverEcuacion(double a, double b, double c) {
        double d = b * b - 4 * a * c;
        System.out.printf("Discriminante (d) = %.4f\n", d);
        if (d > 0) {
            double x1 = (-b + Math.sqrt(d)) / (2 * a);
            double x2 = (-b - Math.sqrt(d)) / (2 * a);
            System.out.println("La ecuación tiene dos soluciones
                               reales distintas:");
            System.out.printf("x1 = %.4f\n", x1);
            System.out.printf("x2 = %.4f\n", x2);
        } else if (Math.abs(d) < 1e-10) {
            double x = -b / (2 * a);
            System.out.println("La ecuación tiene una única solución
                               real (raíz doble):");
            System.out.printf("x = %.4f\n", x);
        } else {
            System.out.println("La ecuación no tiene soluciones reales
                               (el discriminante es negativo).");
        }
    }
}
```

```
}  
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    double a = 0, b = 0, c = 0;  
    while (true) {  
        try {  
            System.out.print("Ingrese coeficiente a: ");  
            a = scanner.nextDouble();  
            if (Math.abs(a) < 1e-10) {  
                System.out.println("Error:El coeficiente 'a' no  
                    puede ser cero (no sería una ecuación  
                    cuadrática).");  
            } else {  
                break;  
            }  
        } catch (InputMismatchException e) {  
            System.err.println("Error: debe introducir un número  
                válido.");  
            scanner.next();  
        }  
    }  
    try {  
        System.out.print("Ingrese coeficiente b: ");  
        b = scanner.nextDouble();  
  
        System.out.print("Ingrese coeficiente c: ");  
        c = scanner.nextDouble();  
    } catch (InputMismatchException e) {  
        System.err.println("Error: la entrada no es válida para b  
            o c.");  
        scanner.close();  
        return;  
    }  
    resolverEcuacion(a, b, c);  
    scanner.close();  
}  
}
```

3.14. *Dados 10 números por teclado sumar los pares y los impares de manera separada y mostrar ambos resultados*

3.14. Datos 10 números por teclado sumar los pares y los impares de manera separada y mostrar ambos resultados

Este ejercicio nos propone que, dados 10 números enteros introducidos por el usuario, los almacene en un array. Posteriormente, deberemos sumar por separado los números pares y los números impares presentes en el array, y finalmente, mostrar ambos resultados (la suma total de pares y la suma total de impares) por pantalla.

```
//fichero sumaparesimpares.java
//Calcula la suma de 10 números pasados por teclado distinguiendo
    entre los pares e impares
import java.util.InputMismatchException;
import java.util.Scanner;
import java.util.Arrays;

public class SumaParesImpares{
    private static final int TAMANO_ARRAY = 10;
    public static void main(String[] args) {
        long[] numeros = new long[TAMANO_ARRAY];
        long sumaPares = 0;
        long sumaImpares = 0;
        Scanner scanner = new Scanner(System.in);
        System.out.printf("Por favor, introduce %d números
            enteros:%n", TAMANO_ARRAY);
        for (int i = 0; i < TAMANO_ARRAY; i++) {
            while (true) {
                System.out.printf("Número %d: ", i + 1);
                try {
                    numeros[i] = scanner.nextLong();
                    break;
                } catch (InputMismatchException e) {
                    System.err.println("Error: Entrada no válida. Debe
                        ser un entero. Intenta de nuevo.");
                    scanner.next();
                }
            }
        }
    }
}
```

```
//Bucle for java para colecciones
for (long numeroActual : numeros) {
    if (numeroActual % 2 == 0) {
        sumaPares += numeroActual;
    } else {
        sumaImpares += numeroActual;
    }
}
System.out.println("\nNúmeros ingresados: " +
    Arrays.toString(numeros));
System.out.printf("Suma total de los números PARES: %d%n",
    sumaPares);
System.out.printf("Suma total de los números IMPARES: %d%n",
    sumaImpares);
scanner.close();
}
}
```

3.15. Función para calcular el factorial de un número (iterativo)

Este ejercicio nos presenta la resolución del cálculo del factorial de un número de manera iterativa. Se ha de tener en cuenta el tipo de entero con el que esté trabajando, ya que el número factorial crece de manera muy rápida.

Nota: Un tipo de dato entero estándar en Java (como `int` o `long`) solo puede contener un valor máximo. La función factorial, $n!$, crece extremadamente rápido. Por ejemplo, $13!$ ya es 6.227.020.800. Cuando $n!$ supera el valor máximo del tipo de dato, se produce un desbordamiento de enteros. Esto hace que el número “dé la vuelta” y se convierta en un valor negativo o incorrecto, lo cual es matemáticamente erróneo. Los **límites prácticos en Java** para un `int` típico de 32 bits (valor máximo alrededor de 2.110^9): $12!$ (479.001.600) cabe. $13!$ (6.227.020.800) provoca un desbordamiento. Para un `long` típico de 64 bits (valor máximo alrededor de 9.210^{18}): $20!$ (alrededor de 2.4310^{18}) cabe. $21!$ (alrededor de 5.110^{19}) provoca un desbordamiento. Al contrario que C y C++, Java soluciona este problema con el tipo de dato `BigInteger` [16], el cual usa almacenamiento dinámico (como una cadena de dígitos grandes), permitiendo representar factoriales gigantes sin pérdida de precisión, en función de los recursos que tenga el equipo.

3.15. Función para calcular el factorial de un número (iterativo)

```
//fichero FactorialI.java
import java.util.InputMismatchException;
import java.util.Scanner;
import java.math.BigInteger;

public class FactorialI {

    // Función que calcula el factorial
    public static BigInteger factorialBigIterativo(int n) {
        // Caso base: 0! y 1! es 1
        if (n == 0 || n == 1) {
            return BigInteger.ONE;
        }
        BigInteger resultado = BigInteger.ONE;
        for (int i = 2; i <= n; i++) {
            resultado = resultado.multiply(BigInteger.valueOf(i));
        }
        return resultado;
    }

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        int numero = 0;
        System.out.print("Introduce un número entero no negativo: ");
        try {
            numero = scanner.nextInt();
        } catch (InputMismatchException e) {
            System.err.println("Error: Entrada no válida. Debe
                               introducir un número entero.");
            scanner.close();
            return;
        }
        scanner.close();
        if (numero < 0) {
            System.err.println("Error: El factorial solo está definido
                               para números no negativos.");
            return;
        }
    }
}
```

```
}
if (numero > 1500) {
    System.out.println("?? Advertencia: El cálculo de
        factoriales de números muy grandes puede tomar un
        tiempo considerable.");
}
BigInteger resultado = factorialBigIterativo(numero);
System.out.printf("El factorial de %d es:%n%s%n", numero,
    resultado.toString());
}
}
```

3.16. Función para calcular el factorial de un número (recursivo)

Este ejercicio nos presenta la resolución del cálculo del factorial de un número de manera recursiva. Aunque ambas versiones (recursiva e iterativa) dan el mismo resultado (hasta que ocurre el overflow). En este caso, hemos puesto un aviso si se produce overflow en la pila, aunque es mucho menos probable con números grandes que en C y C++.

```
//fichero FactorialR.java
import java.util.InputMismatchException;
import java.util.Scanner;
import java.math.BigInteger;

public class FactorialR {

    // Función recursiva que calcula el factorial recursivo
    public static BigInteger factorialBigRecursivo(int n) {
        if (n == 0 || n == 1) {return BigInteger.ONE;}
        return BigInteger.valueOf(n).multiply(factorialBigRecursivo(n
            - 1));
    }

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        int numero;
        System.out.print("Introduce un número entero no negativo: ");
        try {
```

3.16. Función para calcular el factorial de un número (recursivo)

```
        numero = scanner.nextInt();
    } catch (InputMismatchException e) {
        System.err.println("Error: Entrada no válida. Debe
            introducir un número entero.");
        scanner.close();
        return;
    } finally {
        scanner.close();
    }
    if (numero < 0) {
        System.err.println("Error: El factorial solo está definido
            para números no negativos.");
        return;
    }
    if (numero > 1500) {
        System.out.println("Advertencia: Para números muy grandes,
            la recursividad podría fallar por desbordamiento de
            pila.");
    }
    try {
        BigInteger resultado = factorialBigRecursivo(numero);
        System.out.printf("%nEl factorial de %d es:%n%n%s%n",
            numero, resultado.toString());
    } catch (StackOverflowError e) {
        System.err.println("FATAL ERROR: Desbordamiento de la pila
            (StackOverflowError).");
        System.err.printf("El número %d es demasiado grande para
            calcularse de forma recursiva.%n", numero);
        System.err.println("Intente usar un número menor o la
            implementación iterativa.");
    }
}
}
```

3.17. Función para calcular el número combinatorio de N sobre M

Este ejercicio propone realizar el cálculo de un número combinatorio dados los valores N y M por teclado. Para realizar el cálculo de un número combinatorio de N sobre M, vamos a utilizar la conocida como fórmula del coeficiente binomial $\binom{N}{M} = \frac{N!}{M!(N-M)!}$ [4], la cual usa la función realizada en 3.15 del factorial para obtener los resultados. Hay una forma más eficiente de realizar el número combinatorio sin utilizar factoriales, pero no era objetivo de este ejercicio.

```
// fichero Combinatorio.java
// Calcula el número combinatorio C(n, r) o "n sobre r" usando
// factorial iterativo
import java.math.BigInteger;
import java.util.InputMismatchException;
import java.util.Scanner;

public class Combinatorio {

    // Función factorial iterativa
    public static BigInteger factorial(BigInteger n) {
        if (n.equals(BigInteger.ZERO) || n.equals(BigInteger.ONE))
            return BigInteger.ONE;
        BigInteger resultado = BigInteger.ONE;
        for (BigInteger i = BigInteger.TWO;
             i.compareTo(n) <= 0;
             i = i.add(BigInteger.ONE)) {
            resultado = resultado.multiply(i);
        }
        return resultado;
    }

    // Cálculo de C(n, r) = n! / (r! * (n - r)!)
    public static BigInteger combinatorio(int n, int r) {
        if (r < 0 || r > n) return BigInteger.ZERO;
        BigInteger bigN = BigInteger.valueOf(n);
        BigInteger bigR = BigInteger.valueOf(r);
        BigInteger factN = factorial(bigN);
        BigInteger factR = factorial(bigR);
        BigInteger factNR = factorial(bigN.subtract(bigR));
```

3.17. Función para calcular el número combinatorio de N sobre M

```
        return factN.divide(factR.multiply(factNR));
    }
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = 0, r = 0;
        try {
            System.out.print("Introduce un número entero no negativo
                             para n: ");
            n = scanner.nextInt();
            if (n < 0) {
                System.err.println("Error: n debe ser no negativo.");
                return;
            }
            System.out.print("Introduce un número entero no negativo
                             para r: ");
            r = scanner.nextInt();
            if (r < 0) {
                System.err.println("Error: r debe ser no negativo.");
                return;
            }
        } catch (InputMismatchException e) {
            System.err.println("Error: Entrada no válida. Debe
                               introducir un número entero.");
            return;
        } finally {
            scanner.close();
        }
        if (n > 1000) {
            System.out.println("Advertencia: calcular factoriales
                               grandes puede tardar un poco...");
        }
        BigInteger resultado = combinatorio(n, r);
        System.out.printf("%nC(%d, %d) = %s\n", n, r,
                           resultado.toString());
    }
}
```

3.18. Función para resolver $(a + b)^n$ utilizando el desarrollo del binomio de Newton

Este ejercicio nos propone resolver $(a + b)^n$, dados los valores a,b y n. Para ello nos pide utilizar la fórmula del **binomio de Newton** [4]. La fórmula empleada para la resolución es la siguiente [4]: $(a + b)^n = \sum_{k=0}^n \binom{n}{k} a^{n-k} b^k$ donde $\binom{n}{k} = \frac{n!}{k!(n-k)!}$, la cual es la función combinatorio que hemos visto en 3.17. Como dijimos anteriormente, existe otro método para resolver el combinatorio que permite calcular números más grande y binomios más grandes, pero queda fuera de estos ejercicios.

```
// fichero Newton.java
// Calcula la expansión del binomio de Newton (a + b)^n
import java.util.Scanner;
import java.math.BigInteger;

public class Newton {
    // Factorial iterativo
    public static BigInteger factorial(int n) {
        if (n == 0 || n == 1)
            return BigInteger.ONE;

        BigInteger resultado = BigInteger.ONE;
        for (int i = 2; i <= n; i++) {
            resultado = resultado.multiply(BigInteger.valueOf(i));
        }
        return resultado;
    }

    // Cálculo de C(n, r) = n! / (r! * (n - r)!)
    public static BigInteger nCr(int n, int r) {
        if (r < 0 || r > n)
            return BigInteger.ZERO;

        BigInteger factN = factorial(n);
        BigInteger factR = factorial(r);
        BigInteger factNR = factorial(n - r);

        return factN.divide(factR.multiply(factNR));
    }
}
```

3.18. Función para resolver $(a + b)^n$ utilizando el desarrollo del binomio de Newton

```
}

// Expansión del binomio de Newton: (a + b)^n
public static void binomioDeNewton(BigInteger a, BigInteger b, int
n) {
    if (n < 0) {
        System.out.println("Error: El exponente N debe ser no
negativo.");
        return;
    }
    System.out.printf("Expansión de (%s + %s)^d:%n",
        a.toString(), b.toString(), n);
    BigInteger sumaTotal = BigInteger.ZERO;
    for (int k = 0; k <= n; k++) {
        BigInteger coef = nCr(n, k);
        BigInteger potA = a.pow(n - k);
        BigInteger potB = b.pow(k);
        BigInteger termino = coef.multiply(potA).multiply(potB);
        sumaTotal = sumaTotal.add(termino);

        System.out.printf("  + (%s * %s^d * %s^d) = %s%n",
            coef.toString(), a.toString(), n - k,
            b.toString(), k, termino.toString());
    }

    System.out.printf("Valor Total (por suma de términos) = %s%n",
        sumaTotal.toString());
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    try {
        System.out.print("Introduce un número entero para a: ");
        BigInteger a = sc.nextBigInteger();

        System.out.print("Introduce un número entero para b: ");
        BigInteger b = sc.nextBigInteger();
    }
```

```
System.out.print("Introduce un número entero no negativo  
para n: ");  
int n = sc.nextInt();  
  
if (n < 0) {  
    System.out.println("Error: n debe ser no negativo.");  
    return;  
}  
if (n > 1000) {  
    System.out.println("Advertencia: El cálculo puede  
tardar mucho tiempo para n > 1000.");  
}  
  
binomioDeNewton(a, b, n);  
  
} catch (Exception e) {  
    System.out.println("Error: Entrada no válida.");  
} finally {  
    sc.close();  
}  
}
```

3.19. Implementar la función de Fibonacci para valores enteros positivos

Este ejercicio nos propone, dado un número entero positivo, calcular su valor de la sucesión de Fibonacci [4]. Para ello utilizaremos la fórmula matemática de Fibonacci [4]:

$$F(n) = F(n - 1) + F(n - 2)$$

Nota: La implementación recursiva es más directa, siguiendo la definición matemática para Fibonacci, pero es ineficiente para valores grandes de n debido a la repetición de cálculos. Por tanto, implementaremos la función de manera iterativa.

3.19. Implementar la función de Fibonacci para valores enteros positivos

```
// fichero Fibo.java
// Calcula el n-ésimo número de Fibonacci.
import java.util.Scanner;
import java.math.BigInteger;

public class Fibo {

    // Función iterativa para calcular el n-ésimo número de Fibonacci.
    public static BigInteger fibonacciIterativo(int n) {

        if (n < 0) {
            throw new IllegalArgumentException("El índice N debe ser
                no negativo.");
        }

        if (n == 0) {
            return BigInteger.ZERO;
        }

        if (n == 1) {
            return BigInteger.ONE;
        }

        BigInteger a = BigInteger.ZERO;
        BigInteger b = BigInteger.ONE;
        BigInteger resultado = BigInteger.ZERO;

        for (int i = 2; i <= n; i++) {
            resultado = a.add(b);
            a = b;
            b = resultado;
        }
        return resultado;
    }

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
```

```
int n;

System.out.print("Introduce el índice N: ");

try {
    n = scanner.nextInt();
} catch (Exception e) {
    System.err.println("Error: Entrada no válida. Debe
        introducir un número entero.");
    scanner.close();
    return;
}
scanner.close();

if (n < 0) {
    System.err.println("Error: El índice N debe ser no
        negativo.");
    return;
}

if (n > 10000) {
    System.out.printf("Advertencia: El cálculo para N=%d
        puede tardar considerablemente.%n", n);
}

BigInteger resultado = fibonacciIterativo(n);
System.out.printf("%nEl %d-ésimo número de Fibonacci
    (iterativo BigInteger) es:%n%s%n",
        n, resultado.toString());
}
```

3.20. Implementar un procedimiento para seleccionar y visualizar los K bits de orden inferior de un número entero proporcionado.

3.20. Implementar un procedimiento para seleccionar y visualizar los K bits de orden inferior de un número entero proporcionado.

Este ejercicio tiene como objetivo extraer y mostrar por pantalla los K bits menos significativos (los bits de más a la derecha o "últimos bits") de un número entero dado como entrada.

```
// fichero NkBits.java
// Muestra los últimos k bits de un número entero
import java.util.Scanner;
import java.util.InputMismatchException;

public class NkBits{
    public static void mostrarUltimosKBits(long numero, int k) {
        final int TAM_LONG = Long.SIZE;
        if (k <= 0) {
            System.out.println("Error: k debe ser mayor que 0.");
            return;
        }
        if (k > TAM_LONG) {
            System.out.printf("Advertencia: k fue limitado al tamaño
                             del tipo long (%d bits).%n", TAM_LONG);
            k = TAM_LONG;
        }
        System.out.printf("Los últimos %d bits de %d son: ", k,
                           numero);
        for (int i = k - 1; i >= 0; i--) {
            long bit = (numero >>> i) & 1;
            System.out.print(bit);
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Introduce un número entero largo (long):
                             ");
        }
    }
}
```

```
        long num = sc.nextLong();
        System.out.print("Introduce cuántos bits mostrar (k): ");
        int k = sc.nextInt();
        mostrarUltimosKBits(num, k);
    } catch (InputMismatchException e) {
        System.err.println("Error: Entrada no válida. Debe
            introducir valores numéricos válidos.");
    } catch (Exception e) {
        System.err.println("Error inesperado: " + e.getMessage());
    } finally {
        sc.close();
    }
}
```

3.21. Implementar una función que invierta los últimos K bits de orden inferior de un número entero proporcionado

En este ejercicio se pretende realizar una función que, dado un número entero y los k bits últimos que se desean invertir, devuelva el nuevo número. Para ello, la inversión de bits se logra eficientemente creando una máscara donde los últimos K bits están a 1 y el resto a 0, y luego aplicando el operador XOR entre el número y esa máscara.

Creación de la Máscara

$1 \ll k$: Genera un número con el bit en la posición k encendido. Ejemplo con $k = 4$: 00010000 (valor decimal 16)

$(1 \ll k) - 1$: Al restar 1 de cualquier potencia de 2 (o valor con un solo bit encendido), se ponen a 1 todos los bits a la derecha de esa posición. Ejemplo con $k = 4$: 00010000 - 1 = 00001111 (valor decimal 15)

Esta máscara, 00001111, tiene los 4 bits a la derecha listos para la inversión, y los 0's a la izquierda aseguran que el resto del número permanezca inalterado.

Aplicación del XOR

La inversión se realiza mediante el operador $\oplus$ (XOR bit a bit):

Número Original (45): ...00101101

$\oplus$ Máscara ($k = 4$): ...00001111

Resultado (34): ...00100010

3.21. Implementar una función que invierta los últimos K bits de orden inferior de un número entero proporcionado

```
// fichero Mascara.java
// Invierte (cambia 0?1 y 1?0) los últimos k bits de un número entero
import java.util.InputMismatchException;
import java.util.Scanner;

public class Mascara {
    public static long invertirUltimosKBits(long numero, int k) {
        final int TAM_LONG = Long.SIZE; // 64 bits
        if (k <= 0) {
            System.out.println("Error: k debe ser mayor que 0.");
            return numero;
        }
        if (k >= TAM_LONG) {
            return ~numero;
        }
        long mascara = (1L << k) - 1;
        return numero ^ mascara;
    }

    public static void mostrarUltimosKBits(long numero, int k) {
        final int TAM_LONG = Long.SIZE;
        if (k <= 0) return;
        if (k > TAM_LONG) k = TAM_LONG;
        for (int i = k - 1; i >= 0; i--) {
            long bit = (numero >>> i) & 1;
            System.out.print(bit);
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Introduce un número entero largo (long): ");
            long num = sc.nextLong();
            System.out.print("Introduce cuántos bits invertir (k): ");
            int k = sc.nextInt();
            long resultado = invertirUltimosKBits(num, k);
        } catch (InputMismatchException e) {
            System.out.println("Error: introduce un número entero válido.");
        }
    }
}
```

```
        System.out.printf("Número resultante: %d\n", resultado);
        System.out.print("Bits invertidos (últimos " + k + "): ");
        mostrarUltimosKBits(resultado, k);

    } catch (InputMismatchException e) {
        System.err.println("Error: Entrada no válida. Debe
            introducir valores numéricos válidos.");
    } catch (Exception e) {
        System.err.println("Error inesperado: " + e.getMessage());
    } finally {
        sc.close();
    }
}
```

3.22. Invertir una cadena de texto leída por teclado, sin utilizar las funciones del propio lenguaje de programación

Este ejercicio nos propone realizar la inversión de una cadena de texto leída por teclado, pero sin utilizar funciones del lenguaje. Se va a realizar de forma manual, utilizando un bucle y la técnica de intercambio de caracteres (swap), dado que los Strings en Java son inmutables, convertiremos la cadena a un array de caracteres para realizar los intercambios manualmente.

```
// fichero Invierte.java
// Invierte una cadena de texto sin usar métodos del lenguaje

import java.util.Scanner;

public class Invierte {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce una cadena de texto: ");
        String texto = sc.nextLine();
```

3.23. Leer una cadena de caracteres desde teclado, decir longitud y concatenar con otra utilizando las funciones de la biblioteca estándar de Java

```
char[] caracteres = texto.toCharArray();

int inicio = 0;
int fin = caracteres.length - 1;
char temp;

while (inicio < fin) {
    temp = caracteres[inicio];
    caracteres[inicio] = caracteres[fin];
    caracteres[fin] = temp;

    inicio++;
    fin--;
}

String invertida = new String(caracteres);

System.out.println("Cadena original: " + texto);

System.out.println("Cadena invertida: " + invertida);

sc.close();

}
```

3.23. Leer una cadena de caracteres desde teclado, decir longitud y concatenar con otra utilizando las funciones de la biblioteca estándar de Java

Este ejercicio nos propone leer una cadena de caracteres y decir qué longitud tiene y concatenarla con otra cadena, como por ejemplo “txt”. Para ello, utilizaremos los métodos estándar de la clase String, como `.length()` para la longitud y el operador `+` o el método `.concat()` para la unión. Este ejercicio también se puede resolver de manera manual utilizando bucles, pero es mucho menos eficiente.

```
// fichero LeeLongi.java
// Lee una cadena, muestra su longitud y la concatena con ".txt"
import java.util.Scanner;

public class LeeLongi {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String cadena;
        System.out.println("Introduzca cadena:");
        while (true) {
            cadena = sc.nextLine();
            cadena = cadena.stripTrailing(); // elimina saltos de
                línea y espacios al final
            int len = cadena.length();
            if (len == 0) {
                break;
            }
            System.out.printf("La cadena contiene %d caracteres.%n",
                len);
            String cadenaResultado = cadena + ".txt";
            System.out.printf("Concatenación: %s%n", cadenaResultado);
            System.out.println("Introduzca cadena (o solo Enter para
                terminar):");
        }
        sc.close();
    }
}
```

3.24. Convertir cadenas a letras mayúsculas y contar sus vocales

Este ejercicio consiste en leer varias cadenas de caracteres desde teclado, seguidamente convertir a letras mayúsculas y posteriormente contar sus vocales. En él, usaremos tanto cadenas de caracteres como arrays. Para este ejercicio no se utilizan funciones de la librería string de Java.

3.24. Convertir cadenas a letras mayúsculas y contar sus vocales

```
// fichero Conteo.java
// Lee varias frases, las convierte a mayúsculas y cuenta las vocales.

import java.util.Scanner;
public class Conteo {

    private static final int NUM_FRASES = 4;
    private static final int MAX_LONGITUD = 200;

    public static String convertirAMayusculas(String s) {
        StringBuilder resultado = new StringBuilder();
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            resultado.append(Character.toUpperCase(c));
        }
        return resultado.toString();
    }
    // Cuenta las vocales (A, E, I, O, U) en una cadena ya en
    // mayúsculas
    public static int contarVocales(String s) {
        int contador = 0;
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            switch (c) {
                case 'A':
                case 'E':
                case 'I':
                case 'O':
                case 'U':
                    contador++;
                    break;
                default:
                    break;
            }
        }
        return contador;
    }
}
```

```
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);
    String[] listaFrases = new String[NUM_FRASES];
    int totalVocales = 0;

    System.out.printf("Introduce %d frases/líneas de texto:%n",
        NUM_FRASES);
    for (int i = 0; i < NUM_FRASES; i++) {
        System.out.printf("Frase %d: ", i + 1);
        String linea = sc.nextLine();
        if (linea.length() > MAX_LONGITUD) {
            System.out.printf("Advertencia: se truncará a %d
                caracteres.%n", MAX_LONGITUD);
            linea = linea.substring(0, MAX_LONGITUD);
        }
        listaFrases[i] = linea;
    }

    for (int i = 0; i < NUM_FRASES; i++) {
        String frase = listaFrases[i];
        String mayus = convertirAMayusculas(frase);
        int vocales = contarVocales(mayus);
        totalVocales += vocales;
        System.out.printf("Frase %d (MAYÚS): '%s' -> Vocales
            contadas: %d%n",
            i + 1, mayus, vocales);
    }

    System.out.printf("El número total de vocales en todas las
        frases es: %d%n", totalVocales);
    sc.close();
}
```

3.25. Función para realizar la trasposición de una matriz de números enteros dada

En este ejercicio se pretende crear una función para realizar la trasposición de una matriz de números enteros. Para ello vamos a tener en cuenta la siguiente información:

La **trasposición de una matriz** A (denotada como A^T) [2] es una operación que **cambia las filas por las columnas** y viceversa. Si la matriz original A tiene dimensiones $m \times n$ (filas $\times$ columnas), su matriz transpuesta A^T tendrá dimensiones $n \times m$.

El elemento en la posición (i, j) de la matriz original A se convierte en el elemento en la posición (j, i) de la matriz transpuesta A^T [2].

$$A_{i,j} = (A^T)_{j,i}$$

```
// fichero Trasposicion.java
// Trasposición de una matriz de enteros
public class Trasposicion {

    private static final int R_FILAS = 3;
    private static final int C_COLUMNSAS = 4;
    // Función que traspone una matriz de tamaño filas_orig x cols_orig
    public static void trasponerMatriz(int filas_orig, int cols_orig,
        int[][] original, int[][] transpuesta) {
        for (int i = 0; i < filas_orig; i++) {
            for (int j = 0; j < cols_orig; j++) {
                transpuesta[j][i] = original[i][j];
            }
        }
    }

    public static void imprimirMatriz(int filas, int columnas, int[][]
        matriz) {
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < columnas; j++) {
                System.out.printf("%4d", matriz[i][j]);
            }
            System.out.println();
        }
    }

    public static void main(String[] args) {
```

```
int[][] A = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};

int[][] AT = new int[C_COLUMNAS][R_FILAS];

System.out.printf("Matriz Original A (%d x %d)%n", R_FILAS,
    C_COLUMNAS);
imprimirMatriz(R_FILAS, C_COLUMNAS, A);
trasponerMatriz(R_FILAS, C_COLUMNAS, A, AT);
System.out.printf("%nMatriz Transpuesta AT (%d x %d)%n",
    C_COLUMNAS, R_FILAS);
imprimirMatriz(C_COLUMNAS, R_FILAS, AT);
}
}
```

3.26. Función para multiplicar 2 matrices cuadradas de números enteros

En este ejercicio se pretende crear una función para realizar la multiplicación de 2 matrices cuadradas de números enteros.

Para ello, seguiremos la explicación dada en [2], en la cual indica que la multiplicación de dos matrices cuadradas, A y B , de dimensión $N \times N$, da como resultado una matriz C , también de dimensión $N \times N$.

Para multiplicar $A \cdot B$, el número de columnas de A debe ser igual al número de filas de B . Si A y B son ambas $N \times N$, la condición siempre se cumple.

Cada elemento de la matriz resultante, C_{ij} , se calcula como el producto escalar (la suma de productos) de la fila i de A por la columna j de B [2].

$$C_{ij} = \sum_{k=1}^N A_{ik} B_{kj}$$

3.26. Función para multiplicar 2 matrices cuadradas de números enteros

```
// fichero MultiplicaMatrices.java
public class MultiplicaMatrices {

    public static int[][] multiplicarMatricesCuadradas(int[][] A,
        int[][] B) {

        int N = A.length;
        if (N == 0 || A[0].length != N || B.length != N || B[0].length
            != N) {
            System.err.println("Error: Las matrices deben ser
                cuadradas (N x N) y tener el mismo tamaño.");
            return null;
        }
        int[][] C = new int[N][N];
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                C[i][j] = 0;
                for (int k = 0; k < N; k++) {
                    C[i][j] += A[i][k] * B[k][j];
                }
            }
        }
        return C;
    }

    public static void imprimirMatriz(int[][] matriz) {
        if (matriz == null) {
            System.out.println("[Matriz nula]");
            return;
        }
        for (int i = 0; i < matriz.length; i++) {
            for (int j = 0; j < matriz[0].length; j++) {
                System.out.printf("%5d ", matriz[i][j]);
            }
            System.out.println();
        }
    }

    public static void main(String[] args) {
```

```
int[][] A = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12},
    {13, 14, 15, 16}
};
int[][] B = {
    {1, 0, 0, 0},
    {0, 1, 0, 0},
    {0, 0, 1, 0},
    {0, 0, 0, 1}
};
int N_actual = A.length;
System.out.printf("Multiplicación de Matrices Cuadradas (%dx%d
    x %dx%d) ---%n",
                    N_actual, N_actual, N_actual, N_actual);
System.out.println("\nMatriz A:");
imprimirMatriz(A);

System.out.println("\nMatriz B (Identidad):");
imprimirMatriz(B);

int[][] C = multiplicarMatricesCuadradas(A, B);

if (C != null) {
    System.out.println("\nMatriz Resultado C = A * B:");
    imprimirMatriz(C);
}
}
```

Capítulo 4

Manejo de Ficheros y Memoria Dinámica

Aunque los lenguajes de alto nivel como Java abstraen la gestión directa del hardware, siguen ofreciendo herramientas potentes para interactuar directamente con los recursos fundamentales del sistema operativo: la memoria y el almacenamiento persistente (ficheros). Dominar estos dos conceptos es crucial para desarrollar aplicaciones que no sólo sean eficientes en el uso de recursos, sino también capaces de manejar grandes volúmenes de datos y mantener información entre sesiones. Hoy en día está mucho más en desuso la interacción de los lenguajes con ficheros, siendo estas funciones sustituidas por conexiones a bases de datos mediante librerías específicas o APIs. Aun así, es muy socorrido poder manejar ficheros donde guardar información en local. Con respecto a la memoria dinámica, a la hora de manejar datos, estructuras u objetos que necesiten valores más allá de los que proporciona estáticamente el lenguaje, es muy importante manejar las peticiones de memoria al sistema para alojar dichos datos y estructuras. Como ejemplo, podemos tener un array o matriz que queramos que crezca en tiempo de ejecución, ya que no sabemos los datos que va a albergar en tiempo de compilación y necesitamos ir pidiendo memoria conforme crece o decrece. Este capítulo se divide en dos secciones principales, explorando cómo Java nos permite superar las limitaciones de la memoria estática y la volatilidad de los datos en tiempo de ejecución. Para revisar las especificaciones propias del lenguaje, proponemos al lector revisar algunos de los libros más destacados en la bibliografía, tales como [16] [6] [14].

4.1. Crear y Escribir en un fichero

Este ejercicio nos propone crear un fichero de texto y escribir sobre él. Se pretende aprender los comandos básicos para crear un fichero (abrir) y cerrar el fichero.

```
// fichero CreaFiche.java
//Ejemplo para crear/escribir ficheros en Java

import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

public class CreaFiche {
    public static void main(String[] args) {

        try (PrintWriter fichero = new PrintWriter(new
            FileWriter("datos.txt"))) {

            fichero.println("Esta es la primera línea.");

            fichero.printf("El número PI es aproximadamente %.4f%s",
                3.14159, System.lineSeparator());

            fichero.println("Tercera línea de ejemplo.");

        } catch (IOException e) {
            System.err.println("Error al abrir/crear el fichero
                datos.txt: " + e.getMessage());
            System.exit(1);
        }
    }
}
```

4.2. Leer y mostrar contenido de un fichero de texto

Este ejercicio nos propone leer un fichero de texto y mostrar el contenido por pantalla. Para ello, utilizaremos el mismo fichero escrito en el problema anterior 4.1 como entrada.

```
// fichero LeerFiche.java
//Ejemplo para leer ficheros de texto en Java.
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LeerFiche {

    public static void main(String[] args) {

        String linea;
        try (BufferedReader fichero = new BufferedReader(new
            FileReader("datos.txt"))) {

            System.out.println("\nContenido de 'datos.txt':");

            while ((linea = fichero.readLine()) != null) {
                System.out.println(linea);
            }

        } catch (IOException e) {

            System.err.println("Error al abrir el fichero datos.txt: "
                + e.getMessage());
            System.exit(1);
        }

    }
}
```

4.3. Contar caracteres y palabras de un fichero de texto

Este ejercicio nos propone leer el archivo datos.txt (previamente creado en el ejercicio 4.1) y contar el total de caracteres y el número de espacios en blanco (que podemos usar como un estimador de palabras)

```
// fichero ContarCar.java
//Ejemplo para leer un fichero carácter a carácter y contar caracteres
// y palabras.
import java.io.FileReader;
import java.io.IOException;
public class ContarCaracteres {
    public static void main(String[] args) {
        int character;
        int contador_caracteres = 0, contador_palabras = 0;
        boolean dentro_de_palabra = false;
        try (FileReader fichero = new FileReader("datos.txt")) {
            while ((character = fichero.read()) != -1) {
                contador_caracteres++;
                if (Character.isWhitespace((char) character)) {
                    dentro_de_palabra = false;
                } else {
                    if (!dentro_de_palabra) {
                        contador_palabras++;
                        dentro_de_palabra = true;
                    }
                }
            }
        } catch (IOException e) {
            System.err.println("Error al abrir el fichero datos.txt: "
                + e.getMessage());
            System.exit(1);
        }
        System.out.println("Total de caracteres leídos: " +
            contador_caracteres);
        System.out.println("Total de palabras: " + contador_palabras);
    }
}
```

4.4. Guardar una estructura en un fichero binario y cargarla en memoria para ser mostrada por pantalla

4.4. Guardar una estructura en un fichero binario y cargarla en memoria para ser mostrada por pantalla

Este ejercicio nos propone definir una clase, inicializar una instancia y guardarla directamente en un archivo binario (registro.dat) usando la serialización de objetos (ObjectOutputStream), para luego leerla recuperando el objeto (ObjectInputStream).

```
// fichero CrearLeerBinario.java
//Programa que crea y lee un archivo binario con un registro de alumno.

import java.io.*;

public class CrearLeerBinario {

    static class RegistroAlumno implements Serializable {

        int id;
        String nombre;
        float nota;

        RegistroAlumno(int id, String nombre, float nota) {
            this.id = id;
            this.nombre = nombre;
            this.nota = nota;
        }
    }

    private static final String NOMBRE_FICHERO = "registro.dat";

    static void crearBinario() {

        RegistroAlumno alumno = new RegistroAlumno(101, "Sofia Perez",
            9.5f);

        try (ObjectOutputStream salida = new ObjectOutputStream(new
            FileOutputStream(NOMBRE_FICHERO))) {
```

```
        salida.writeObject(alumno);
        System.out.println("Registro guardado correctamente en '"
            + NOMBRE_FICHERO + "'.");
    } catch (IOException e) {
        System.err.println("Error al escribir el fichero binario: "
            + e.getMessage());
    }
}

static void leerBinario() {
    try (ObjectInputStream entrada = new ObjectInputStream(new
        FileInputStream(NOMBRE_FICHERO))) {

        RegistroAlumno alumnoLeido = (RegistroAlumno)
            entrada.readObject();
        System.out.printf("Datos cargados del fichero: ID=%d,
            Nombre=%s, Nota=%.2f%n",
                alumnoLeido.id, alumnoLeido.nombre,
                alumnoLeido.nota);

    } catch (IOException | ClassNotFoundException e) {
        System.err.println("Error al leer el fichero binario: " +
            e.getMessage());
        System.exit(1);
    }
}

public static void main(String[] args) {
    crearBinario();
    leerBinario();
}
}
```

4.5. Insertar al final de un fichero de texto

Este ejercicio nos propone, dado el fichero de texto `datos.txt` creado en 4.1, escribir una nueva línea al final del mismo.

```
// fichero EscribeFinal.java
// Ejercicio para escribir al final de un fichero de texto una nueva
// línea.
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

public class EscribeFinal {

    public static void main(String[] args) {

        String nombrefichero = "datos.txt";

        try (PrintWriter fichero = new PrintWriter(new
            FileWriter(nombrefichero, true))) {

            archivo.println("Esta es la línea AÑADIDA al final
                (Append).");
            System.out.println("\nSe ha añadido una línea al final de
                '" + nombrefichero + "'.");

        } catch (IOException e) {
            System.err.println("Error al abrir el fichero " +
                nombrefichero + " en modo append: " + e.getMessage());
            System.exit(1);
        }

    }
}
```

4.6. Asignación y liberación básica de memoria con vectores

Este ejercicio nos propone, reservar memoria para un vector de N enteros, inicializarlo, imprimirlo y luego liberar la memoria. Para ello utilizaremos la primitiva `new` que ofrece Java y el recolector de basura.

Nota: Este ejercicio está pensado para ver el manejo de la primitiva `new` para pedir memoria de manera tradicional sobre tipos de datos existentes en el lenguaje y no objetos creados para trabajar con ellos. Obviamente, en Java existen ya estructura de datos orientadas a objetos para poder tener arrays dinámicos en tiempo de ejecución, las cuales puedan crecer y puedan decrecer sin mayor esfuerzo que el de llamar a la función correspondiente dentro del objeto, por ejemplo `ArrayList`.

```
// fichero AsignaLibera.java
public class AsignaLibera {

    public static void main(String[] args) {

        int N = 5;
        int[] vector=null;
        try{
            vector = new int[N];
        }catch (OutOfMemoryError e) {
            System.out.println("Error: No hay suficiente memoria para
                               crear el array.");
            System.exit(1);
        }
        System.out.println("Vector Dinámico de " + N + " Elementos");
        for (int i = 0; i < N; i++) {
            vector[i] = i * 10;
            System.out.print(vector[i] + " ");
        }
        System.out.println();
        System.out.println("Memoria liberada automáticamente por el
                           Garbage Collector.");
    }
}
```

4.7. Trabajo con Matrices Dinámicas 2D

Este ejercicio nos propone crear una matriz $M \times N$ (2D) dinámica y asegurar la correcta liberación de memoria.

```
// fichero M2D.java

public class M2D {
    public static void main(String[] args) {

        int M = 3;
        int N = 4;
        int[][] matriz = null;
        try {
            matriz = new int[M][N];
        } catch (OutOfMemoryError e) {
            System.out.println("Error: No hay suficiente memoria para
                               crear la matriz.");
            System.exit(1);
        }
        for (int i = 0; i < M; i++) {
            for (int j = 0; j < N; j++) {
                matriz[i][j] = i * N + j + 1;
            }
        }
        System.out.println("\nMatriz Dinámica " + M + "x" + N);
        for (int i = 0; i < M; i++) {
            for (int j = 0; j < N; j++) {
                System.out.printf("%3d ", matriz[i][j]);
            }
            System.out.println();
        }

        System.out.println("Memoria de la matriz liberada
                           automáticamente por el Garbage Collector.");
    }
}
```

Además del método que acabamos de ver, como dijimos en 4.6, podemos utilizar la clase `ArrayList` para implementar dichas matrices. Vamos a ver cómo se haría el ejercicio usando dicha clase.

```
// fichero M2D2.java
import java.util.ArrayList;

public class M2D2 {
    public static void main(String[] args) {

        int M = 3;
        int N = 4;
        ArrayList<ArrayList<Integer>> matriz = new ArrayList<>();

        try {
            for (int i = 0; i < M; i++) {
                ArrayList<Integer> fila = new ArrayList<>();
                for (int j = 0; j < N; j++) {
                    fila.add(i * N + j + 1);
                }
                matriz.add(fila);
            }
        } catch (OutOfMemoryError e) {
            System.out.println("Error: No hay suficiente memoria para
                               crear la matriz.");
            System.exit(1);
        }

        System.out.println("\nMatriz Dinámica con ArrayList " + M +
                           "x" + N);
        for (ArrayList<Integer> fila : matriz) {
            for (Integer valor : fila) {
                System.out.printf("%3d ", valor);
            }
            System.out.println();
        }
    }
}
```

4.8. Aumento del tamaño de un vector en tiempo de ejecución

Este ejercicio nos propone, asignar un bloque de memoria para un vector, llenarlo con datos y luego usar la función `copyOf` para aumentar su tamaño, conservando los datos existentes y agregando nuevos elementos. En Java, para estas operaciones para vectores, como venimos comentando, se usa la clase `ArrayList` y ésta se encarga de manejar todos los procesos internos de crear, añadir memoria y liberar.

```
//fichero AumentarVector.java
import java.util.Arrays;

public class AumentarVector {
    public static void main(String[] args) {

        int N_inicial = 3;
        int N_nuevo = 5;
        int[] vector = null;

        try {
            vector = new int[N_inicial];
        } catch (OutOfMemoryError e) {
            System.out.println("Error: No hay suficiente memoria para
                               crear el vector.");
            System.exit(1);
        }
        for (int i = 0; i < N_inicial; i++) {
            vector[i] = i + 10;
        }

        System.out.println("\nVector Inicial (" + N_inicial + "
                           elementos:");
        for (int val : vector) {
            System.out.print(val + " ");
        }
        System.out.println();

        try {
            vector = Arrays.copyOf(vector, N_nuevo);
```

```
    } catch (OutOfMemoryError e) {  
        System.out.println("Error: No hay suficiente memoria para  
            redimensionar el vector.");  
        System.exit(1);  
    }  
  
    for (int i = N_inicial; i < N_nuevo; i++) {  
        vector[i] = i + 100;  
    }  
  
    System.out.println("Vector Redimensionado (" + N_nuevo + "  
        elementos):");  
    for (int val : vector) {  
        System.out.print(val + " ");  
    }  
    System.out.println();  
    System.out.println("Memoria liberada automáticamente por el  
        Garbage Collector.");  
}
```

Capítulo 5

Algoritmos de ordenación y búsqueda

En este capítulo vamos a realizar un repaso a los algoritmos más destacados de la literatura sobre ordenación y búsqueda [13] [8] [20]. Dichos algoritmos vienen a mejorar los métodos tradicionales para abordar este tipo de problemas y ayudan a mejorar los tiempos de ejecución, haciendo nuestros programas mucho más eficientes en coste de tiempo y recursos. Cuando trabajamos con datos, es muy importante que las estructuras que utilicemos tengan unos tiempos de búsqueda eficientes, para ello, normalmente, se suelen utilizar métodos avanzados de inserción en la estructura de datos y de búsqueda. Muchas veces el método de inserción hace uso de métodos de ordenación para conseguir esa eficiencia en las búsquedas, de ahí la importancia de ver estos algoritmos utilizados en problemas. A pesar de que en Java la mayoría de los algoritmos vienen ya implementados en la librería estándar, hemos preferido realizar una revisión pedagógica de los mismos utilizando las bases del lenguaje.

Nota: En estos ejercicios hemos asumido que cuando trabajamos con enteros, vamos a utilizar el tipo `int`, ya que no se van a hacer grandes operaciones aritméticas que necesiten del uso de `long` como en capítulos anteriores. Para el uso de `long`, o de otro tipo de dato como `float` o `double`, simplemente habría que realizar cambios en el tipo de variable que se utiliza, ya que toda la mecánica de los algoritmos sería idéntica. Aunque usamos `int`, en entornos reales con grandes volúmenes de datos o valores muy altos, deberíamos considerar el desbordamiento de enteros y usar `long`.

5.1. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo de la Burbuja

Este ejercicio nos propone, dado un conjunto de números enteros desordenado, devolver el conjunto ordenado utilizando el **algoritmo de ordenación de la burbuja**, también conocido como **Bubble Sort** [13]. La simplicidad del algoritmo tiene un costo en eficiencia. La complejidad temporal del Método de la Burbuja es: $O(n^2)$. Donde n es el número de elementos [13]. Esto lo hace poco práctico para conjuntos de datos grandes en comparación con otros algoritmos más eficientes como *Quick Sort* o *Merge Sort*.

```
// fichero Burbuja.java
public class Burbuja {
    public static void ordenarBurbuja(int[] arr) {
        int n = arr.length;
        int temp;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }

    public static void main(String[] args) {
        int[] numeros = {10, 7, 8, 9, 1, 5};
        System.out.print("\nConjunto original: ");
        for (int num : numeros) {
            System.out.print(num + " ");
        }
        System.out.println();
        ordenarBurbuja(numeros);
        System.out.print("Conjunto ordenado (Bubble Sort): ");
        for (int num : numeros) { System.out.print(num + " "); }
        System.out.println();
    }
}
```

5.2. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo Quicksort

5.2. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo Quicksort

Este ejercicio nos propone, dado un conjunto de números desordenado, devolver el conjunto ordenado utilizando el **algoritmo de ordenación Quicksort** [20] [13]. Quicksort es un algoritmo de ordenación extremadamente eficiente y ampliamente utilizado. Es conocido por ser uno de los más rápidos en la práctica para grandes conjuntos de datos. En el caso **Promedio y Mejor Caso**, su orden de complejidad es de $O(n \log n)$ y en el **peor de los casos** es de $O(n^2)$ [20]. Este último caso resulta en subproblemas muy desequilibrados ($n - 1$ y 0), forzando al algoritmo a realizar un número de comparaciones similar al del *Bubble Sort*. En este ejercicio haremos la implementación del algoritmo Quicksort.

La función `qsort` está disponible en Java para arrays de tipos primitivos (como `int[]`, `double[]`, `char[]`, etc.). Java utiliza una implementación de Quicksort particular. Específicamente, desde Java 7, utiliza un algoritmo llamado *Dual-Pivot Quicksort* [17]. Esta es una versión mejorada del Quicksort clásico (como el que se ha implementado en este ejercicio) que generalmente es más rápida. Si se desea utilizar ordenación para arrays de objetos, se utilizará también la función `sort`, pero ésta no implementa el método `Qsort` ni *Dual-Pivot Qsort*, utiliza *TimSort* [15], que es un algoritmo híbrido muy eficiente que combina Merge Sort (Ordenación por mezcla) e Insertion Sort (Ordenación por inserción).

```
// fichero QuickSort.java

public class QSort {

    // Función de intercambio (swap)
    public static void swap(int[] arr, int i, int j) {
        int t = arr[i];
        arr[i] = arr[j];
        arr[j] = t;
    }

    // Función de partición
    public static int particion(int[] arr, int bajo, int alto) {
        int pivote = arr[alto];
        int i = bajo - 1;

        for (int j = bajo; j <= alto - 1; j++) {
            if (arr[j] <= pivote) {
                i++;
            }
        }
    }
}
```

```

        swap(arr, i, j);
    }
}
swap(arr, i + 1, alto);
return i + 1;
}
// Función recursiva Quicksort
public static void quicksort(int[] arr, int bajo, int alto) {
    if (bajo < alto) {
        int pi = particion(arr, bajo, alto);
        quicksort(arr, bajo, pi - 1);
        quicksort(arr, pi + 1, alto);
    }
}
public static void main(String[] args) {

    int[] datos = {10, 70, 8, 90, 1000, 5};

    System.out.print("Vector original: ");
    for (int val : datos) {
        System.out.print(val + " ");
    }
    System.out.println();
    quicksort(datos, 0, datos.length - 1);
    System.out.print("Vector ordenado con Quicksort: ");
    for (int val : datos) {
        System.out.print(val + " ");
    }
    System.out.println();
}
}

```

5.3. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo Quicksort utilizando el método propio de Java para tipos primitivos

5.3. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo Quicksort utilizando el método propio de Java para tipos primitivos

Este ejercicio nos propone, dado un conjunto de números desordenados, devolver el conjunto ordenado utilizando el **algoritmo de ordenación Quicksort** definido en Java para tipos de datos primitivos (Dual-Pivot Quicksort), tal y como hemos explicado en el ejercicio anterior 5.2.

```
/* Fichero QSort2.java */
import java.util.Arrays;

public class QSort2 {

    public static void main(String[] args) {

        int[] numeros = {50, 10, 8, 20, 40};

        System.out.print("Vector antes de sort: ");
        for (int val : numeros) {
            System.out.print(val + " ");
        }
        System.out.println();

        Arrays.sort(numeros);

        System.out.print("Vector después de sort: ");
        for (int val : numeros) {
            System.out.print(val + " ");
        }
        System.out.println();
    }
}
```

5.4. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo Mergesort

Este ejercicio nos propone, dado un conjunto de números desordenado, devolver el conjunto ordenado utilizando el **Algoritmo de ordenación de Mergesort** [20]. **MergeSort** es otro algoritmo de ordenación muy eficiente, está basado en el principio de "Divide y Vencerás"[13]. A diferencia de Quicksort, su rendimiento es consistente: siempre $O(n \log n)$ en el mejor, promedio y peor caso [13].

```
// fichero MergeSort.java
public class MergeSort {
    //Función auxiliar para combinar (merge) dos sub-arrays ordenados.
    //Esta función es la parte "Combinar" del "Divide y Vencerás".
    public static void merge(int[] arr, int inicio, int medio, int
        fin) {

        int n1 = medio - inicio + 1;
        int n2 = fin - medio;
        int total_temp = n1 + n2;
        int[] temp_arr = new int[total_temp];

        for (int i = 0; i < n1; i++) {
            temp_arr[i] = arr[inicio + i];
        }
        for (int j = 0; j < n2; j++) {
            temp_arr[n1 + j] = arr[medio + 1 + j];
        }
        int i = 0;
        int j = n1;
        int k = inicio;
        while (i < n1 && j < total_temp) {
            if (temp_arr[i] <= temp_arr[j]) {
                arr[k] = temp_arr[i];
                i++;
            } else {
                arr[k] = temp_arr[j];
                j++;
            }
        }
    }
}
```

5.4. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo Mergesort

```
        }
        k++;
    }
    while (i < n1) {
        arr[k] = temp_arr[i];
        i++;
        k++;
    }
    while (j < total_temp) {
        arr[k] = temp_arr[j];
        j++;
        k++;
    }
}

//Función recursiva principal de MergeSort.
public static void mergeSort(int[] arr, int inicio, int fin) {
    if (inicio < fin) {
        int medio = inicio + (fin - inicio) / 2;
        mergeSort(arr, inicio, medio);
        mergeSort(arr, medio + 1, fin);
        merge(arr, inicio, medio, fin);
    }
}

public static void main(String[] args) {
    int[] datos = {38, 27, 43, 3, 9, 82, 10};
    System.out.println("Vector original desordenado:");
    for (int val : datos) {
        System.out.print(val + " ");
    }
    System.out.println();
    mergeSort(datos, 0, datos.length - 1);
    System.out.println("\nVector ordenado con MergeSort:");
    for (int val : datos) {
        System.out.print(val + " ");
    }
    System.out.println();
}
}
```

5.5. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo Shellsort

Este ejercicio nos propone, dado un conjunto de números desordenado, devolver el conjunto ordenado utilizando el **algoritmo de ordenación de ShellSort** [20].

El rendimiento del algoritmo **Shellsort** depende crucialmente de la elección de la secuencia de saltos o intervalos h . Una de las secuencias más eficientes y comunes, que optimiza significativamente el tiempo de ejecución en el peor caso, es la propuesta por D. Knuth [8].

La secuencia de intervalos de Knuth [8] se define recursivamente mediante la fórmula $h_{k+1} = 3h_k + 1$. El punto de partida de la secuencia es $h_1 = 1$. Los primeros términos de esta secuencia son: 1, 4, 13, 40, 121, 364, 1093, ...

Utilizar la secuencia de Knuth [8] proporciona al Shellsort una complejidad temporal significativamente mejor que las secuencias triviales (como $h = N/2$): Se ha demostrado que con esta secuencia, el Shellsort alcanza un rendimiento cercano a $O(n^{3/2})$ en el peor caso, o incluso mejor, en torno a $O(n \log^2 n)$, superando con creces la complejidad $O(n^2)$ de los algoritmos de ordenación simples.

```
// fichero ShellSort.java
public class ShellSort {
    //Implementación del algoritmo Shellsort usando la secuencia de
    //Knuth (3*h + 1).
    public static void shellsortKnuth(int[] arr) {
        int n = arr.length, h = 1;
        while (h < n / 3) {
            h = 3 * h + 1;
        }
        while (h >= 1) {
            for (int i = h; i < n; i++) {
                int temp = arr[i];
                int j = i;
                while (j >= h && arr[j - h] > temp) {
                    arr[j] = arr[j - h];
                    j -= h;
                }
                arr[j] = temp;
            }
            h = h / 3;
        }
    }
}
```

5.6. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo RadixSort

```
}  
public static void main(String[] args) {  
    int[] datos = {90, 8, 70, 6, 50, 4, 30, 2, 10, 0, 85, 15, 65,  
        35};  
    System.out.println("Array original desordenado:");  
    for (int val : datos) {System.out.print(val + " ");}  
    System.out.println();  
    shellsortKnuth(datos);  
    System.out.println("\nArray ordenado con Shellsort (Knuth):");  
    for (int val : datos) {  
        System.out.print(val + " ");  
    }  
    System.out.println();  
}
```

5.6. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo RadixSort

Este ejercicio nos propone, dado un conjunto de números desordenado, devolver el conjunto ordenado utilizando el **algoritmo de ordenación de RadixSort** [20] [8].

Radix Sort (ordenación por base o por dígitos), es un algoritmo de ordenación no comparativo que organiza los elementos procesando sus dígitos de forma secuencial [20]. En lugar de comparar directamente los valores entre sí (como hace, por ejemplo, *Quicksort* o *Mergesort*), este algoritmo clasifica los números según cada posición decimal (unidades, decenas, centenas, etc.), utilizando un algoritmo de ordenación **estable** en cada paso, normalmente *Counting Sort*. *Counting Sort* (ordenación por conteo) [20] [8], es un algoritmo no comparativo que se utiliza para ordenar números enteros dentro de un rango limitado. En lugar de comparar elementos entre sí, cuenta cuántas veces aparece cada valor, y a partir de esas frecuencias reconstruye el array ordenado. La complejidad temporal es: $O(d \cdot (n + k))$ y la complejidad espacial es: $O(n + k)$. El algoritmo *Radix Sort* es un método de ordenación eficiente para números enteros cuando los valores son de longitud limitada. Su rendimiento cercano a $O(n)$ lo convierte en una buena opción para grandes volúmenes de datos, siempre que la base y la longitud de los números no sean demasiado grandes [20] [8].

```
// fichero RadixSort.java
import java.util.Arrays;

public class RadixSort {
    //Función auxiliar: obtiene el máximo valor del array.
    public static int getMax(int[] arr) {
        int n = arr.length;
        if (n == 0) return 0;
        int max = arr[0];
        for (int i = 1; i < n; i++) {
            if (arr[i] > max) {
                max = arr[i];
            }
        }
        return max;
    }

    // Algoritmo Counting Sort estable (Ordenación por Conteo).
    public static void countingSort(int[] arr, int exp) {
        int n = arr.length; int[] output = new int[n]; int[] count = new
        int[10];
        for (int i = 0; i < n; i++) {
            int digit = (arr[i] / exp) % 10;
            count[digit]++;
        }
        for (int i = 1; i < 10; i++) {
            count[i] += count[i - 1];
        }
        for (int i = n - 1; i >= 0; i--) {
            int digit = (arr[i] / exp) % 10;
            output[count[digit] - 1] = arr[i];
            count[digit]--;
        }
        for (int i = 0; i < n; i++) {
            arr[i] = output[i];
        }
    }

    public static void radixSort(int[] arr) {
```

5.7. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo HeapSort

```
int n = arr.length;
if (n == 0) return;
int max = getMax(arr);
for (int exp = 1; max / exp > 0; exp *= 10) {
    countingSort(arr, exp);
}

public static void main(String[] args) {
    int[] arr = {170, 45, 75, 90, 802, 24, 2, 66};
    System.out.println("Array original:");
    for (int val : arr) {
        System.out.print(val + " ");
    }
    System.out.println();
    radixSort(arr);
    System.out.println("\nArray ordenado:");
    for (int val : arr) {
        System.out.print(val + " ");
    }
    System.out.println();
}
```

5.7. Dado un conjunto de números enteros desordenado, ordenar mediante el algoritmo HeapSort

Este ejercicio nos propone, dado un conjunto de números enteros desordenado, devolver el conjunto ordenado utilizando el **algoritmo de ordenación de HeapSort** [20]. **Heapsort** (Ordenación por Montículo) es un algoritmo de ordenación por comparación que utiliza la estructura de datos **montículo binario (heap)** para ordenar elementos. Es un algoritmo eficiente, con una complejidad temporal de $\mathcal{O}(n \log n)$ en el mejor, promedio y peor caso. Heapsort se basa en la propiedad de un **Max-Heap** (Montículo Máximo). Un Max-Heap [20] es un árbol binario completo donde:

- El **valor de cada nodo es mayor o igual** que los valores de sus hijos.
- El **elemento más grande** de todo el montículo siempre está en la **raíz** del árbol.

```
// fichero HeapSort.java
public class HeapSort {
    public static void swap(int[] arr, int i, int j) {
        int t = arr[i];
        arr[i] = arr[j];
        arr[j] = t;
    }
    // Función Heapify: restaura la propiedad de Max-Heap en un
        subárbol
    //con raíz en 'i' dentro de un array de tamaño 'n'.
    public static void heapify(int[] arr, int n, int i) {

        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        if (left < n && arr[left] > arr[largest]) {
            largest = left;
        }
        if (right < n && arr[right] > arr[largest]) {
            largest = right;
        }
        if (largest != i) {
            swap(arr, i, largest);
            heapify(arr, n, largest);
        }
    }
    public static void heapSort(int[] arr) {
        int n = arr.length;

        for (int i = n / 2 - 1; i >= 0; i--) {
            heapify(arr, n, i);
        }
        for (int i = n - 1; i > 0; i--) {
            swap(arr, 0, i);
            heapify(arr, i, 0);
        }
    }
}
```

5.8. Dado un conjunto de números enteros, realizar la búsqueda de un número mediante el algoritmo de Búsqueda Lineal

```
public static void main(String[] args) {
    int[] arr = {12, 11, 13, 5, 6, 7};
    System.out.println("Array original:");
    for (int val : arr) {
        System.out.print(val + " ");
    }
    System.out.println();
    heapSort(arr);
    System.out.println("Array ordenado con Heapsort:");
    for (int val : arr) {
        System.out.print(val + " ");
    }
    System.out.println();
}
```

5.8. Dado un conjunto de números enteros, realizar la búsqueda de un número mediante el algoritmo de Búsqueda Lineal

Este ejercicio nos propone, dado un conjunto de números enteros, buscar un número dentro del conjunto utilizando el **algoritmo de búsqueda lineal**. La Búsqueda Lineal (o secuencial) [13] es el algoritmo de búsqueda más simple. Su complejidad es de orden **lineal** ($O(n)$) porque el tiempo de ejecución crece directamente con el número de elementos (n) del array. La complejidad es $O(1)$ si el elemento se encuentra en la **primera posición**. En promedio, se espera buscar $\frac{n}{2}$ elementos, lo que sigue siendo proporcional a n , que es el peor de los casos. Para conjuntos de datos grandes y **ordenados**, la **Búsqueda Binaria** es mucho más eficiente, con una complejidad temporal de $O(\log n)$.

```
// fichero BusquedaLineal.java
public class BusquedaLineal {
    public static int busquedaLineal(int[] arr, int n, int objetivo) {
        for (int i = 0; i < n; i++) {
            if (arr[i] == objetivo) {return i;}
        }
        return -1;
    }
}
```

```

public static void main(String[] args) {
    int[] datos = {10, 5, 20, 15, 8, 30};
    int n = datos.length;
    int objetivo1 = 15;
    int indice;
    System.out.println("Conjunto de datos: {10, 5, 20, 15, 8, 30}");
    indice = busquedaLineal(datos, n, objetivo1);
    if (indice != -1) {
        System.out.printf("Resultado para %d: Encontrado en el índice %d.\n", objetivo1, indice);
    } else {
        System.out.printf("Resultado para %d: No encontrado.\n", objetivo1);
    }
}
}

```

5.9. Dados un conjunto de números enteros ordenados, realizar la búsqueda de un número mediante el algoritmo de Búsqueda Binaria

Este ejercicio nos propone, dado un conjunto de números ordenados, buscar un número perteneciente al conjunto utilizando el **algoritmo de búsqueda binaria** [13]. La Búsqueda Binaria utiliza la estrategia de Divide y Vencerás [13]. La principal ventaja de la Búsqueda Binaria sobre la Búsqueda Lineal es su velocidad. La complejidad temporal es logarítmica, lo que significa que el tiempo de ejecución crece muy lentamente a medida que aumenta el tamaño de los datos, siendo esta $O(\log n)$. Java ya incluye una implementación nativa de búsqueda binaria en la clase utilitaria `java.util.Arrays` (`Arrays.binarySearch()`) [16]. Es muy eficiente, y se puede usar directamente sin necesidad de programarla desde cero.

```

// fichero BusquedaBinaria.java/
public class BusquedaBinaria {
    public static int busquedaBinaria(int[] arr, int objetivo) {
        int bajo = 0;
        int alto = arr.length - 1;

        while (bajo <= alto) {

```

5.9. Dados un conjunto de números enteros ordenados, realizar la búsqueda de un número mediante el algoritmo de Búsqueda Binaria

```
        int medio = bajo + (alto - bajo) / 2;
        if (arr[medio] == objetivo) {
            return medio;
        } else if (arr[medio] < objetivo) {
            bajo = medio + 1;
        } else {
            alto = medio - 1;
        }
    }
    return -1;
}

public static void main(String[] args) {
    int[] datosOrdenados = {5, 8, 12, 15, 20, 30, 40};
    int objetivo1 = 20;
    int objetivo2 = 25;

    System.out.println("Conjunto de datos (Ordenado): {5, 8, 12,
        15, 20, 30, 40}");
    int indice = busquedaBinaria(datosOrdenados, objetivo1);
    if (indice != -1) {
        System.out.printf("Resultado para %d: Encontrado en el índice %d.%n", objetivo1, indice);
    } else {
        System.out.printf("Resultado para %d: No encontrado.%n", objetivo1);
    }
    indice = busquedaBinaria(datosOrdenados, objetivo2);
    if (indice != -1) {
        System.out.printf("Resultado para %d: Encontrado en el índice %d.%n", objetivo2, indice);
    } else {
        System.out.printf("Resultado para %d: No encontrado.%n", objetivo2);
    }
}
```

5.10. Dados un conjunto de números enteros no negativos ordenados, realizar la búsqueda de un número mediante el algoritmo de Búsqueda por Interpolación

Este ejercicio nos propone, dado un conjunto de números enteros no negativos ordenado, buscar un número perteneciente al conjunto utilizando el **algoritmo de búsqueda por interpolación** [20] [8]. Éste es un algoritmo de búsqueda diseñado para conjuntos de datos ordenados que asume una distribución uniforme de los elementos. Es una mejora de la Búsqueda Binaria, ya que no solo divide el espacio de búsqueda a la mitad, sino que estima la posición del elemento objetivo basándose en su valor.

En el caso promedio, y cuando los datos están uniformemente distribuidos, la Búsqueda por Interpolación es más rápida que la Búsqueda Binaria ($O(\log \log n)$ vs $O(\log n)$). En el peor de los casos son idénticos $O(n)$.

```
// fichero BInterpola.java
public class BInterpola {
    //Realiza la Búsqueda por Interpolación en un array ordenado
    //uniformemente distribuido.
    public static int busquedaInterpolacion(int[] arr, int n, int
    objetivo) {
        int bajo = 0;
        int alto = n - 1;
        int pos;
        while (bajo <= alto && objetivo >= arr[bajo] && objetivo <=
        arr[alto]) {
            if (bajo == alto) {
                if (arr[bajo] == objetivo) {
                    return bajo;
                }
                return -1;
            }
            if (arr[alto] == arr[bajo]) {
                if (arr[bajo] == objetivo)
                    return bajo;
                else
                    return -1;
            }
        }
    }
}
```

5.10. Dados un conjunto de números enteros no negativos ordenados, realizar la búsqueda de un número mediante el algoritmo de Búsqueda por Interpolación

```
// Fórmula de interpolación
pos = bajo + (int)((double)(alto - bajo) / (arr[alto] -
    arr[bajo])) * (objetivo - arr[bajo]);
if (pos < 0 || pos >= n) {
    return -1;
}
if (arr[pos] == objetivo) {
    return pos;
}
if (arr[pos] < objetivo) {
    bajo = pos + 1;
} else {
    alto = pos - 1;
}
}
return -1;
}

public static void main(String[] args) {
    int[] datosOrdenados = {10, 20, 30, 40, 50, 60, 70, 80, 90,
        100};
    int n = datosOrdenados.length;
    int objetivo1 = 70;
    System.out.println("Conjunto de datos (Ordenado):{10,20,30,...,100}");
    int indice = busquedaInterpolacion(datosOrdenados, n,
        objetivo1);
    if (indice != -1) {
        System.out.printf("Resultado para %d: Encontrado en el índice %d.%n",
            objetivo1, indice);
    } else {
        System.out.printf("Resultado para %d: No encontrado.%n",
            objetivo1);
    }
}
}
```

Además de todos los algoritmos revisados anteriormente, existen otros algoritmos de búsqueda sobre estructuras de datos más complejas como árboles o grafos, los cuales quedan fuera de este libro de problemas básicos. Para más información sobre algoritmos avanzados véase [20] [13] [8].

Capítulo 6

Trabajo con problemas Concurrentes y Distribuidos

En este capítulo vamos a ver ejercicios que resuelven problemas de concurrencia en el acceso a recursos compartidos y distribuidos. Para ello, vamos a centrarnos en los problemas clásicos que aborda la literatura [21][18], como el problema del productor-consumidor, en el cual existen productores y consumidores compartiendo un recurso y deben poder acceder concurrentemente y en exclusión. Además de dicho problema, veremos el problema del lector-escriptor, en el cual los lectores acceden simultáneamente pero compiten por la exclusión mutua con los escritores. Por último, serán revisados los problemas de paso de mensajes tanto síncronos como asíncronos. Todos estos problemas serán abordados utilizando 4 métodos posibles tales como semáforos, monitores, paso de mensajes síncrono y asíncrono [21][18].

6.1. Semáforos: Problema del productor-consumidor

Este ejercicio nos propone resolver el problema del productor-consumidor utilizando semáforos para acceso concurrente a un mismo buffer compartido circular y sincronización. En este caso hemos decidido utilizar la forma clásica de resolver este problema, pero java ofrece ya directamente una clase llamada `ArrayBlockingQueue` [16] que ya implementa internamente este mismo patrón (productor-consumidor con búfer circular y sincronización).

```
// fichero Spc.java
import java.util.LinkedList;
import java.util.Queue;
import java.util.concurrent.Semaphore;
/**
 * Buffer con control de acceso usando semáforos.
 * Los semáforos controlan explícitamente el número de espacios
 * disponibles y elementos presentes en el buffer.*/
class BufferConSemaforos {
    private final Queue<Integer> buffer;
    private final Semaphore espaciosDisponibles;
    private final Semaphore elementosDisponibles;
    private final Semaphore mutexBuffer;

    public BufferConSemaforos(int capacidad) {
        this.buffer = new LinkedList<>();
        this.espaciosDisponibles = new Semaphore(capacidad);
        this.elementosDisponibles = new Semaphore(0);
        this.mutexBuffer = new Semaphore(1);
    }
    /**
     * Producir usando semáforos para control de concurrencia.
     */
    public void producir(int item) throws InterruptedException {
        // Esperar por un espacio disponible
        espaciosDisponibles.acquire();
        // Entrar en sección crítica
        mutexBuffer.acquire();
        try {
            buffer.add(item);
            System.out.println(Thread.currentThread().getName() +
```

6.1. Semáforos: Problema del productor-consumidor

```
                " - Producido: " + item + " (Buffer: " +
                    buffer.size() + ")");
    } finally {
        // Salir de sección crítica
        mutexBuffer.release();
    }
    // Señalar que hay un nuevo elemento disponible
    elementosDisponibles.release();
}
/**
 * Consumir usando semáforos para control de concurrencia.
 */
public int consumir() throws InterruptedException {
    // Esperar por un elemento disponible
    elementosDisponibles.acquire();
    int item;
    // Entrar en sección crítica
    mutexBuffer.acquire();
    try {
        item = buffer.poll();
        System.out.println(Thread.currentThread().getName() +
            " - Consumido: " + item + " (Buffer: " +
                buffer.size() + ")");
    } finally {
        // Salir de sección crítica
        mutexBuffer.release();
    }
    // Señalar que hay un nuevo espacio disponible
    espaciosDisponibles.release();
    return item;
}
}

public class Spc{
    public static void main(String[] args) {
        System.out.println("\n EJEMPLO: USANDO SEMÁFOROS\n");
        final int CAPACIDAD = 5;
        final int ITEMS_TOTALES = 10;
        BufferConSemaforos buffer = new BufferConSemaforos(CAPACIDAD);
```

```
// Productores con semáforos
Thread productor = new Thread(() -> {
    try {
        for (int i = 1; i <= ITEMS_TOTALES; i++) {
            Thread.sleep((long)(Math.random() * 800));
            buffer.producir(i);
        }
        System.out.println(Thread.currentThread().getName() +
            " - ¡Producción completada!");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}, "Productor-Semáforo");

// Consumidor con semáforos
Thread consumidor = new Thread(() -> {
    try {
        for (int i = 0; i < ITEMS_TOTALES; i++) {
            int item = buffer.consumir();
            Thread.sleep((long)(Math.random() * 1200));
        }
        System.out.println(Thread.currentThread().getName() +
            " - ¡Consumo completado!");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}, "Consumidor-Semáforo");

productor.start();
consumidor.start();

try {
    productor.join();
    consumidor.join();
} catch (InterruptedException e) {
    e.printStackTrace();
}

System.out.println("\n¡Ejemplo completado!\n");
}
```

6.2. Semáforos: Problema de los lectores escritores

Este ejercicio nos propone resolver el problema clásico de Lectores-Escritores utilizando semáforos para acceso concurrente a un mismo fichero de texto. Para evitar el problema de la inanición del escritor, en este caso se ha decidido utilizar prioridad para el escritor (**Algoritmo de Courtois**)[18].

```
import java.util.concurrent.Semaphore;

/**
 * Recurso compartido con control de acceso Lector-Escritor y
 * Prioridad para Escritores.
 * * Lógica de Prioridad Escritor:
 * - Cuando un escritor quiere escribir, bloquea la entrada de nuevos
 *   lectores (usando entradaLectores).
 * - Los lectores que ya estaban leyendo terminan normalmente.
 * - Una vez que no quedan lectores ni escritores previos, el escritor
 *   accede.*/
class BaseDeDatos {

    private int dato = 0; // El recurso compartido

    // Semáforos y Contadores
    private int lectoresActivos = 0;
    private int escritoresEsperando = 0;
    // Mutex para proteger las variables contadoras
    private final Semaphore mutexLectores = new Semaphore(1);
    private final Semaphore mutexEscritores = new Semaphore(1);
    // Semáforo para acceso exclusivo al recurso (escritura)
    private final Semaphore recurso = new Semaphore(1);
    // Semáforo puerta para priorizar escritores (bloquea nuevos
    //   lectores)
    private final Semaphore entradaLectores = new Semaphore(1);
    /**
     * Escribir: Prioridad alta. Cierra la puerta a nuevos lectores.
     */
    public void escribir(int nuevoValor) throws InterruptedException {
        // Anunciamos que hay un escritor esperando
        mutexEscritores.acquire();
```

```
    escritoresEsperando++;
    if (escritoresEsperando == 1) {
        // Si soy el primer escritor esperando, cierro la puerta a
        // los lectores
        entradaLectores.acquire();
    }
    mutexEscritores.release();

    // Espero posesión exclusiva del recurso
    recurso.acquire();
    try {
        // SECCIÓN CRÍTICA DE ESCRITURA
        this.dato = nuevoValor;
        System.out.println(Thread.currentThread().getName() +
            " -Escribiendo: " + dato + " (Lectores: "
            + lectoresActivos + ", Escritores
            + esp: " + escritoresEsperando + ")");
    } finally {
        // Libero recurso
        recurso.release();
    }

    // Salida del escritor
    mutexEscritores.acquire();
    escritoresEsperando--;
    if (escritoresEsperando == 0) {
        // Si no quedan más escritores esperando, abro la puerta a
        // los lectores
        entradaLectores.release();
    }
    mutexEscritores.release();
}

/**
 * Leer: Debe pedir permiso (entradaLectores) por si hay
 * escritores.
 */
public int leer() throws InterruptedException {
    // Pasar por la puerta (si hay escritores esperando, aquí se
```

6.2. Semáforos: Problema de los lectores escritores

```
        bloquea)
    entradaLectores.acquire();
    entradaLectores.release(); // Liberar inmediatamente para que
        pasen otros lectores (si no hay escritores)

    // Protocolo de entrada de lectores
    mutexLectores.acquire();
    lectoresActivos++;
    if (lectoresActivos == 1) {
        // El primer lector bloquea el recurso para que nadie
            escriba mientras leemos
        recurso.acquire();
    }
    mutexLectores.release();
    int lectura = -1;
    try {
        // SECCIÓN CRÍTICA DE LECTURA (Técnicamente concurrente
            con otros lectores)
        lectura = this.dato;
        System.out.println(Thread.currentThread().getName() +
            " - Leído: " + lectura + " (Lectores
                activos: " + lectoresActivos + ")");
    } finally {
        // Protocolo de salida de lectores
        mutexLectores.acquire();
        lectoresActivos--;
        if (lectoresActivos == 0) {
            // El último lector libera el recurso para los
                escritores
            recurso.release();
        }
        mutexLectores.release();
    }
    return lectura;
}

}

public class Sle {
    public static void main(String[] args) {
```

```
System.out.println("\nPROBLEMA LECTOR-ESCRITOR (PRIORIDAD
    ESCRITOR)\n");

final int NUM_LECTORES = 5;
final int NUM_ESCRITORES = 2;
BaseDeDatos bd = new BaseDeDatos();
// Hilos Lectores
for (int i = 1; i <= NUM_LECTORES; i++) {
    new Thread(() -> {
        try {
            while (true) {
                Thread.sleep((long)(Math.random() * 1000));
                bd.leer();
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }, "Lector-" + i).start();
}
// Hilos Escritores
for (int i = 1; i <= NUM_ESCRITORES; i++) {
    new Thread(() -> {
        try {
            while (true) {
                Thread.sleep((long)(Math.random() * 2000)); //
                Escriben con menos frecuencia
                int valorAleatorio = (int)(Math.random() *
                    100);
                bd.escribir(valorAleatorio);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }, "Escritor-" + i).start();
}
}
```

6.3. Semáforos: Problema de la cena de los filósofos

Este ejercicio nos propone resolver el problema clásico de la cena de los filósofos mediante semáforos. En este caso vamos a plantearlo como sigue. Cinco filósofos ($N=5$) se sientan alrededor de una mesa circular. Entre cada par de filósofos hay un palillo (tenedor). Por lo tanto, hay 5 palillos. Cada filósofo alterna entre dos estados: pensar y comer. Para comer, un filósofo necesita coger los dos palillos que tiene a su lado (el izquierdo y el derecho). Un filósofo solo puede coger un palillo a la vez. Para evitar el deadlock, implementaremos una solución que restringe el número de filósofos que pueden intentar comer simultáneamente, también conocida como **solución del árbitro o camarero**[21] [18].

```
// fichero Scf.java
import java.util.concurrent.Semaphore;
import java.util.Random;

/**
 * La clase Mesa gestiona los recursos compartidos (palillos y la sala)
 */
class Mesa {

    public static final int N_FILOSOFOS = 5;
    public final Semaphore[] palillos = new Semaphore[N_FILOSOFOS];
    public final Semaphore sala = new Semaphore(N_FILOSOFOS - 1);

    public Mesa() {
        for (int i = 0; i < N_FILOSOFOS; i++) {
            palillos[i] = new Semaphore(1);
        }
    }
}

/**
 * El hilo Filosofo, que alterna entre pensar y comer.
 */
class Filosofo extends Thread {

    private final int id;
    private final Mesa mesa;
    private final Random random = new Random();
```

```
private final int palilloIzquierdo;
private final int palilloDerecho;

public Filosofo(int id, Mesa mesa) {
    this.id = id;
    this.mesa = mesa;

    this.palilloIzquierdo = id;
    this.palilloDerecho = (id + 1) % Mesa.N_FILOSOFOS;
}

private void pensar() throws InterruptedException {
    System.out.println("Filósofo " + id + " está PENSANDO.");
    Thread.sleep(random.nextInt(2000) + 1000);
}

private void comer() throws InterruptedException {
    System.out.println("Filósofo " + id + " está COMIENDO.");
    Thread.sleep(random.nextInt(1500) + 1000);
}

@Override
public void run() {
    try {
        while (true) {

            pensar();

            System.out.println("Filósofo " + id + " tiene HAMBRE y
                               quiere entrar al comedor...");
            mesa.sala.acquire();
            System.out.println("Filósofo " + id + " entró al
                               comedor.");

            mesa.palillos[palilloIzquierdo].acquire();
            System.out.println("Filósofo " + id + " cogió palillo
                               izq (" + palilloIzquierdo + ")");
```

6.3. Semáforos: Problema de la cena de los filósofos

```
        Thread.sleep(100);

        mesa.palillos[palilloDerecho].acquire();
        System.out.println("Filósofo " + id + " cogió palillo
            der (" + palilloDerecho + ")");

        comer();

        System.out.println("Filósofo " + id + " terminó de
            comer. Suelta palillos.");
        mesa.palillos[palilloDerecho].release();
        mesa.palillos[palilloIzquierdo].release();

        mesa.sala.release();
        System.out.println("Filósofo " + id + " salió del
            comedor.");
    }
} catch (InterruptedException e) {
    System.out.println("Filósofo " + id + " fue
        interrumpido.");
}
}
}

public class Scf {

    public static void main(String[] args) {
        System.out.println("Iniciando la Cena de los Filósofos...");
        Mesa mesa = new Mesa();

        for (int i = 0; i < Mesa.N_FILOSOFOS; i++) {
            new Filosofo(i, mesa).start();
        }
    }
}
```

6.4. Monitores: Problema del productor-consumidor

Este ejercicio nos propone resolver el problema del productor-consumidor utilizando un monitor para acceso concurrente a un mismo buffer compartido circular y sincronización.

```
// fichero Mpc.java
import java.util.LinkedList;
import java.util.Queue;

class BufferCompartido {
    private final Queue<Integer> buffer;
    private final int capacidad;

    public BufferCompartido(int capacidad) {
        this.buffer = new LinkedList<>();
        this.capacidad = capacidad;
    }

    /**
     * Método sincronizado para producir un elemento.
     * El productor espera si el buffer está lleno.*/
    public synchronized void producir(int item) throws
        InterruptedException {
        // Esperar mientras el buffer esté lleno
        // Usamos while (no if) para proteger contra "spurious wakeups"
        while (buffer.size() == capacidad) {
            System.out.println(Thread.currentThread().getName() +
                               " - Buffer lleno. Productor
                               esperando...");
            wait(); // Libera el lock y espera
        }
        // Agregar el item al buffer
        buffer.add(item);
        System.out.println(Thread.currentThread().getName() +
                           " - Producido: " + item + " (Buffer: " +
                           buffer.size() + "/" + capacidad + ")");

        // Notificar a los consumidores que hay datos disponibles
        notifyAll(); // Despierta a todos los hilos en espera
    }
}
```

6.4. Monitores: Problema del productor-consumidor

```
/**
 * Método sincronizado para consumir un elemento.
 * El consumidor espera si el buffer está vacío.*/
public synchronized int consumir() throws InterruptedException {
    // Esperar mientras el buffer esté vacío
    while (buffer.isEmpty()) {
        System.out.println(Thread.currentThread().getName() +
            " - Buffer vacío. Consumidor esperando...");
        wait();
    }
    // Extraer el item del buffer
    int item = buffer.poll();
    System.out.println(Thread.currentThread().getName() +
        " - Consumido: " + item + " (Buffer: " +
        buffer.size() + "/" + capacidad + ")");
    // Notificar a los productores que hay espacio disponible
    notifyAll();
    return item;
}
}
/**
 * Clase Productor que genera elementos y los coloca en el buffer.
 */
class Productor implements Runnable {
    private final BufferCompartido buffer;
    private final int itemsAProducir;

    public Productor(BufferCompartido buffer, int itemsAProducir) {
        this.buffer = buffer;
        this.itemsAProducir = itemsAProducir;
    }
    @Override
    public void run() {
        try {
            for (int i = 1; i <= itemsAProducir; i++) {
                // Simular tiempo de producción
                Thread.sleep((long)(Math.random() * 1000));
            }
        }
    }
}
```

```
        buffer.producir(i);
    }
    System.out.println(Thread.currentThread().getName() + " - i
        Producción completada!");
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    System.err.println("Productor interrumpido");
}
}
}
}
/**
 * Clase Consumidor que extrae elementos del buffer y los procesa.
 */
class Consumidor implements Runnable {
    private final BufferCompartido buffer;
    private final int itemsAConsumir;

    public Consumidor(BufferCompartido buffer, int itemsAConsumir) {
        this.buffer = buffer;
        this.itemsAConsumir = itemsAConsumir;
    }
    @Override
    public void run() {
        try {
            for (int i = 0; i < itemsAConsumir; i++) {
                int item = buffer.consumir();
                // Simular tiempo de procesamiento
                Thread.sleep((long)(Math.random() * 1500));
            }
            System.out.println(Thread.currentThread().getName() + " - i
                Consumo completado!");
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.err.println("Consumidor interrumpido");
        }
    }
}
```

6.4. Monitores: Problema del productor-consumidor

```
public class Mpc{
    public static void main(String[] args) {

        System.out.println("\nEJEMPLO: MONITOR USANDO WAIT/NOTIFY\n");

        final int CAPACIDAD = 5;
        final int ITEMS_TOTALES = 10;

        BufferCompartido buffer = new BufferCompartido(CAPACIDAD);

        // Crear 2 productores y 2 consumidores
        Thread productor1 = new Thread(new Productor(buffer,
            ITEMS_TOTALES/2), "Productor-1");
        Thread productor2 = new Thread(new Productor(buffer,
            ITEMS_TOTALES/2), "Productor-2");
        Thread consumidor1 = new Thread(new Consumidor(buffer,
            ITEMS_TOTALES/2), "Consumidor-1");
        Thread consumidor2 = new Thread(new Consumidor(buffer,
            ITEMS_TOTALES/2), "Consumidor-2");

        // Iniciar todos los hilos
        productor1.start();
        productor2.start();
        consumidor1.start();
        consumidor2.start();

        // Esperar a que terminen
        try {
            productor1.join();
            productor2.join();
            consumidor1.join();
            consumidor2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("\nEjemplo completado!\n");
    }
}
```

6.5. Monitores: Problema de los lectores escritores

Este ejercicio nos propone resolver el problema clásico de Lectores-Escritores utilizando un monitor para acceso concurrente a un mismo fichero de texto. Para evitar el problema de la inanición del escritor, en este caso se ha decidido utilizar prioridad para el escritor (**Algoritmo de Courtois**) [18].

```
// fichero Mle.java
/**
 * Monitor para el problema Lector-Escritor con Prioridad a
 * Escritores.*/
class BaseDeDatosMonitor {
    private int dato = 0;

    // Variables de estado del Monitor
    private int lectoresActivos = 0;
    private boolean escribiendo = false;
    private int escritoresEsperando = 0; // Crucial para la prioridad
    /**
     * Protocolo de entrada para lectores.
     * Si hay un escritor escribiendo 0 escritores esperando, el
     * lector se bloquea.
     */
    private synchronized void iniciarLectura() throws
        InterruptedException {
        // PRIORIDAD ESCRITOR: Si hay alguien escribiendo 0 hay
        // escritores en la fila
        while (escribiendo || escritoresEsperando > 0) {
            wait();
        }
        lectoresActivos++;
    }
    /**
     * Protocolo de salida para lectores.
     */
    private synchronized void terminarLectura() {
        lectoresActivos--;
        // Si soy el último lector, aviso a los hilos esperando
        // (posibles escritores)
    }
}
```

6.5. Monitores: Problema de los lectores escritores

```
        if (lectoresActivos == 0) {
            notifyAll();
        }
    }

    /**
     * Protocolo de entrada para escritores.
     */
    private synchronized void iniciarEscritura() throws
        InterruptedException {
        // Me anoto como esperando para bloquear a nuevos lectores
        (Prioridad)
        escritoresEsperando++;

        try {
            // Mientras haya alguien leyendo o escribiendo, espero
            while (lectoresActivos > 0 || escribiendo) {
                wait();
            }
        } finally {
            // Ya voy a dejar de esperar y pasar a escribir (o si
            salta excepción)
            escritoresEsperando--;
        }

        escribiendo = true;
    }

    /**
     * Protocolo de salida para escritores.
     */
    private synchronized void terminarEscritura() {
        escribiendo = false;
        notifyAll(); // Despierto a todos (lectores o escritores) para
            que compitan
    }

    public void leer() {
        try {
            iniciarLectura();
            try {
```

```
// SECCIÓN CRÍTICA (Lectura) - Fuera del synchronized
// para permitir concurrencia
System.out.println(Thread.currentThread().getName() +
    " - Leído: " + dato + " (Lectores
    activos: " + lectoresActivos +
    ")");
// Simulamos tiempo de lectura
Thread.sleep((long) (Math.random()* 1000));
} finally {
    terminarLectura();
}
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
}

public void escribir(int valor) {
    try {
        iniciarEscritura();
        try {
            // SECCIÓN CRÍTICA (Escritura)
            this.dato = valor;
            System.out.println(Thread.currentThread().getName() +
                " - Escribiendo: " + dato + "
                (Lectores: " + lectoresActivos +
                ", Escritores esp: " +
                escritoresEsperando + ")");

            // Simulamos tiempo de escritura
            Thread.sleep((long) (Math.random() * 1500));
        } finally {
            terminarEscritura();
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}

}

public class Mle {
    public static void main(String[] args) {
```

6.5. Monitores: Problema de los lectores escritores

```
System.out.println("\n--- LECTOR-ESCRITOR CON MONITORES
(PRIORIDAD ESCRITOR) ---\n");

final int NUM_LECTORES = 5;
final int NUM_ESCRITORES = 2;
BaseDeDatosMonitor bd = new BaseDeDatosMonitor();

// Hilos Lectores
for (int i = 1; i <= NUM_LECTORES; i++) {
    new Thread(() -> {
        try {
            while (true) {
                Thread.sleep((long)(Math.random() * 1000));
                bd.leer();
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }, "Lector-" + i).start();
}

// Hilos Escritores
for (int i = 1; i <= NUM_ESCRITORES; i++) {
    new Thread(() -> {
        try {
            while (true) {
                Thread.sleep((long)(Math.random() * 2000));
                int valorAleatorio = (int)(Math.random() *
                    100);
                bd.escribir(valorAleatorio);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }, "Escritor-" + i).start();
}
}
```

6.6. Monitores: Problema de la cena de los filósofos

Este ejercicio nos propone resolver el problema clásico de la cena de los filósofos mediante un monitor. En este caso vamos a plantearlo como sigue. Cinco filósofos ($N=5$) se sientan alrededor de una mesa circular. Entre cada par de filósofos hay un palillo (tenedor). Por lo tanto, hay 5 palillos. Cada filósofo alterna entre dos estados: pensar y comer. Para comer, un filósofo necesita coger los dos palillos que tiene a su lado (el izquierdo y el derecho). Un filósofo solo puede coger un palillo a la vez. Para evitar el deadlock, implementaremos una solución que restringe el número de filósofos que pueden intentar comer simultáneamente, también conocida como **solución del árbitro o camarero**[21] [18].

```
//fichero Mfc.java
import java.util.Random;

/**
 * Monitor que implementa la solución del CAMARERO (Árbitro).
 * Clave: Limita el número de comensales sentados a (N-1) para evitar
 * interbloqueo.
 */
class MesaCamarero {

    public static final int N_FILOSOFOS = 5;
    // Semáforo lógico (aforo): Máximo 4 filósofos sentados a la vez
    private static final int AFORO_MAXIMO = N_FILOSOFOS - 1;

    private final boolean[] palillosLibres = new boolean[N_FILOSOFOS];
    private int comensalesSentados = 0; // Contador de gente sentada a
    la mesa

    public MesaCamarero() {
        for (int i = 0; i < N_FILOSOFOS; i++) {
            palillosLibres[i] = true;
        }
    }

    /**
     * Intenta coger los palillos.
     * 1. Primero pide permiso para sentarse (si hay 4 sentados,
     * espera).
```

6.6. Monitores: Problema de la cena de los filósofos

```
* 2. Luego intenta coger los palillos (si están ocupados, espera).
*/
public synchronized void cogerPalillos(int id) throws
    InterruptedException {
    int izq = id;
    int der = (id + 1) % N_FILOSOFOS;

    // Si la mesa está casi llena (4 personas), el 5º se queda de
    // pie esperando.
    while (comensalesSentados >= AFORO_MAXIMO) {
        System.out.println("    Filósofo " + id + " ESPERA DE PIE
            (Mesa llena: " + comensalesSentados + " comensales).");
        wait();
    }
    // Si pasa aquí, se sienta
    comensalesSentados++;
    System.out.println("Filósofo " + id + " se sienta a la mesa
        (Hay " + comensalesSentados + " sentados).");

    // Ahora que está sentado, intenta coger los tenedores.
    while (!palillosLibres[izq] || !palillosLibres[der]) {
        System.out.println("    Filósofo " + id + " espera
            cubiertos (Sentado, pero sin tenedores).");
        wait();
    }

    // Reserva los tenedores
    palillosLibres[izq] = false;
    palillosLibres[der] = false;

    System.out.println("Filósofo " + id + " COGIÓ los palillos " +
        izq + " y " + der);
}

/**
 * Suelta los palillos y se levanta de la mesa.
 */
public synchronized void soltarPalillos(int id) {
```

```
int izq = id;
int der = (id + 1) % N_FILOSOFOS;

// Libera tenedores
palillosLibres[izq] = true;
palillosLibres[der] = true;

comensalesSentados--; // Deja un hueco libre en la mesa

System.out.println("Filósofo " + id + " soltó palillos y SE
    LEVANTÓ (Quedan " + comensalesSentados + " sentados).");

// Avisa a todos:
// - A los que esperan de pie (wait del aforo)
// - A los que esperan cubiertos (wait de palillos)
notifyAll();
}
}

class Filosofo extends Thread {
    private final int id;
    private final MesaCamarero mesa;
    private final Random random = new Random();

    public Filosofo(int id, MesaCamarero mesa) {
        this.id = id;
        this.mesa = mesa;
    }

    private void pensar() throws InterruptedException {
        System.out.println("Filósofo " + id + " está PENSANDO ...");
        Thread.sleep(random.nextInt(1000) + 500);
    }

    private void comer() throws InterruptedException {
        System.out.println("Filósofo " + id + " está COMIENDO ...");
        Thread.sleep(random.nextInt(1000) + 500);
    }
}
```

6.6. Monitores: Problema de la cena de los filósofos

```
@Override
public void run() {
    try {
        while (true) {
            pensar();
            System.out.println("Filósofo " + id + " tiene
                               HAMBRE.");

            mesa.cogerPalillos(id);
            comer();
            mesa.soltarPalillos(id);
        }
    } catch (InterruptedException e) {
        System.out.println("Filósofo " + id + " fue
                           interrumpido.");
    }
}

}

public class Mcf {
    public static void main(String[] args) {
        System.out.println("EJEMPLO CENA DE LOS FILÓSOFOS (SOLUCIÓN
                           DEL CAMARERO/AFORO) ");
        System.out.println(" Mesa para " + MesaCamarero.N_FILOSOFOS +
                           " filósofos.");
        System.out.println(" El camarero solo deja sentarse a " +
                           (MesaCamarero.N_FILOSOFOS - 1) + " a la vez.\n");

        MesaCamarero mesa = new MesaCamarero();

        // Creamos y arrancamos los hilos
        for (int i = 0; i < MesaCamarero.N_FILOSOFOS; i++) {
            new Filosofo(i, mesa).start();
        }
    }
}
```

6.7. Paso de Mensajes Asíncrono. El Problema del productor consumidor

Este ejercicio nos propone resolver el problema de los productores-consumidores utilizando paso de mensajes asíncronos para acceso concurrente a un mismo buffer compartido circular y sincronización. En este caso utilizaremos `ArrayBlockingQueue` [16] para trabajar con el paso de mensajes y buzones.

```
// fichero PmApc.java
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ArrayBlockingQueue;
/**
 * Clase que representa al hilo Productor.
 * Envía "mensajes" (los ítems) a un buzón.*/
class Productor extends Thread {
    private BlockingQueue<Integer> buzon;
    public Productor(BlockingQueue<Integer> buzon) {
        this.buzon = buzon;
    }
    @Override
    public void run() {
        int item = 0;
        try {
            while (true) {
                item++;
                Thread.sleep((int) (Math.random() * 1000));
                System.out.println("Productor produce y envía: " +
                    item);
                buzon.put(item);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
/**
 * Clase que representa al hilo Consumidor.
 * Recibe "mensajes" (los ítems) del buzón.*/
```

6.7. Paso de Mensajes Asíncrono. El Problema del productor consumidor

```
class Consumidor extends Thread {
    private BlockingQueue<Integer> buzon;
    public Consumidor(BlockingQueue<Integer> buzon) {
        this.buzon = buzon;
    }
    @Override
    public void run() {
        try {
            while (true) {

                int item = buzon.take();
                System.out.println("Consumidor recibe y consume: " +
                    item);

                Thread.sleep((int) (Math.random() * 1500));
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

public class PmApc {
    public static void main(String[] args) {

        final int CAPACIDAD_BUZON = 5;
        BlockingQueue<Integer> buzon = new
            ArrayBlockingQueue<>(CAPACIDAD_BUZON);

        Productor p1 = new Productor(buzon);
        p1.start();
        Consumidor c1 = new Consumidor(buzon);
        c1.start();
        Productor p2 = new Productor(buzon);
        p2.start();
        Consumidor c2 = new Consumidor(buzon);
        c2.start();
    }
}
```

6.8. Paso de Mensajes Asíncrono: Problema de los lectores escritores

Este ejercicio nos propone resolver el problema de los lectores-escritores utilizando paso de mensajes asíncronos para acceso concurrente a un mismo buffer compartido circular y sincronización. En este caso utilizaremos `ArrayBlockingQueueE` [16] y prioridad para el escritor (**Algoritmo de Courtois**) [18] para trabajar con el paso de mensajes y buzones.

```
// fichero PmAle.java
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.Queue;
import java.util.LinkedList;
//Este es el recurso compartido. La seguridad la garantiza el
    Controlador, que solo da permiso
// a los hilos para acceder en el momento correcto.
class FicheroSimulado {

    private String contenido = "[Contenido inicial del fichero]";

    public void leer(String nombre) {
        System.out.println(nombre + " está LEYENDO: " +
            this.contenido);
        try {
            Thread.sleep((long) (Math.random() * 500));
        } catch (InterruptedException e) {}
    }

    public void escribir(String nombre) {
        System.out.println(nombre + " está ESCRIBIENDO...");
        try {
            Thread.sleep((long) (Math.random() * 1500));
        } catch (InterruptedException e) {}
        this.contenido = "[Escrito por " + nombre + "]";
        System.out.println("... " + nombre + " terminó de escribir.");
    }
}
```

6.8. Paso de Mensajes Asíncrono: Problema de los lectores escritores

```
// Interfaz base para todos los mensajes
interface Mensaje {}

// Mensajes de Lector
class PeticionAbrirLectura implements Mensaje {

    public final String nombre;
    public final BlockingQueue<Boolean> canalRespuesta;
    public PeticionAbrirLectura(String n, BlockingQueue<Boolean> c) {
        nombre = n; canalRespuesta = c; }
}

class PeticionCerrarLectura implements Mensaje {

    public final String nombre;
    public PeticionCerrarLectura(String n) { nombre = n; }
}

// Mensajes de Escritor
class PeticionAbrirEscritura implements Mensaje {

    public final String nombre;
    public final BlockingQueue<Boolean> canalRespuesta;
    public PeticionAbrirEscritura(String n, BlockingQueue<Boolean> c)
        { nombre = n; canalRespuesta = c; }
}

class PeticionCerrarEscritura implements Mensaje {

    public final String nombre;
    public PeticionCerrarEscritura(String n) { nombre = n; }
}

//Este es el "proceso Controlador". Gestiona el estado (nl) y las
//colas de peticiones.
class Controlador extends Thread {

    private final BlockingQueue<Mensaje> buzonPeticiones = new
        LinkedBlockingQueue<>();
    private int lectoresActivos = 0;
    private boolean escritorActivo = false;
    private final Queue<PeticionAbrirLectura> lectoresEnEspera = new
        LinkedList<>();
}
```

```
private final Queue<PeticionAbrirEscritura> escritoresEnEspera =
    new LinkedList<>();
public Controlador() {
    this.start();
}
public void enviarPeticion(Mensaje msg) throws
    InterruptedException {
    buzonPeticiones.put(msg);
}
@Override
public void run() {
    System.out.println("Controlador de bloqueos iniciado.");
    try {
        while (true) {
            Mensaje msg = buzonPeticiones.take();
            if (msg instanceof PeticionAbrirLectura) {
                procesarAbrirLectura((PeticionAbrirLectura) msg);
            }
            else if (msg instanceof PeticionCerrarLectura) {
                procesarCerrarLectura((PeticionCerrarLectura) msg);
            }
            else if (msg instanceof PeticionAbrirEscritura) {
                procesarAbrirEscritura((PeticionAbrirEscritura)
                    msg);
            }
            else if (msg instanceof PeticionCerrarEscritura) {
                procesarCerrarEscritura((PeticionCerrarEscritura)
                    msg);
            }
        }
    } catch (InterruptedException e) {
        System.out.println("Controlador interrumpido.");
    }
}
private void procesarAbrirLectura(PeticionAbrirLectura p) throws
    InterruptedException {

    if (escritoresEnEspera.size() > 0 || escritorActivo) {
```

6.8. Paso de Mensajes Asíncrono: Problema de los lectores escritores

```
        System.out.println("    Controlador: " + p.nombre + " en
            espera (Prioridad Escritor).");
        lectoresEnEspera.add(p);
    } else {
        lectoresActivos++;
        p.canalRespuesta.put(true);
    }
}

private void procesarCerrarLectura(PeticionCerrarLectura p) throws
    InterruptedException {

    lectoresActivos--; // n1 := n1 - 1
    if (lectoresActivos == 0) {
        intentarDespertarEscritor();
    }
}

private void procesarAbrirEscritura(PeticionAbrirEscritura p)
    throws InterruptedException {

    if (lectoresActivos > 0 || escritorActivo) {
        System.out.println("    Controlador: " + p.nombre + " en
            espera (Recurso ocupado).");
        escritoresEnEspera.add(p);
    } else {
        escritorActivo = true;
        p.canalRespuesta.put(true);
    }
}

private void procesarCerrarEscritura(PeticionCerrarEscritura p)
    throws InterruptedException {

    escritorActivo = false;
    if (!intentarDespertarEscritor()) {
        intentarDespertarLectores();
    }
}

private boolean intentarDespertarEscritor() throws
    InterruptedException {
```

```
        if (escritoresEnEspera.size() > 0 && lectoresActivos == 0 &&
            !escritorActivo) {
            PeticionAbrirEscritura p = escritoresEnEspera.remove();
            System.out.println("Controlador: Despertando a escritor "
                               + p.nombre);
            escritorActivo = true;
            p.canalRespuesta.put(true);
            return true;
        }
        return false;
    }

    private void intentarDespertarLectores() throws
        InterruptedException {

        if (escritoresEnEspera.size() == 0 && !escritorActivo) {
            while (!lectoresEnEspera.isEmpty()) {
                PeticionAbrirLectura p = lectoresEnEspera.remove();
                System.out.println("Controlador: Despertando a lector
                                   " + p.nombre);
                lectoresActivos++;
                p.canalRespuesta.put(true);
            }
        }
    }
}

//Hilo Lector (Cliente)
class Lector extends Thread {

    private final FicheroSimulado fichero;
    private final Controlador controlador;
    private final String nombre;
    private final BlockingQueue<Boolean> miBuzon = new
        ArrayBlockingQueue<>(1);

    public Lector(FicheroSimulado f, Controlador c, String n) {

        this.fichero = f;
```

6.8. Paso de Mensajes Asíncrono: Problema de los lectores escritores

```
        this.controlador = c;
        this.nombre = n;
    }
    @Override
    public void run() {
        try {
            while (true) {

                System.out.println(nombre + " pide permiso para
                    leer...");
                controlador.enviarPetición(new
                    PeticiónAbrirLectura(nombre, miBuzon));
                miBuzon.take();
                System.out.println(nombre + " obtuvo permiso.");
                fichero.leer(nombre);
                controlador.enviarPetición(new
                    PeticiónCerrarLectura(nombre));
                Thread.sleep((long) (Math.random() * 2000));
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

//Hilo Escritor (Cliente)
class Escritor extends Thread {

    private final FicheroSimulado fichero;
    private final Controlador controlador;
    private final String nombre;
    private final BlockingQueue<Boolean> miBuzon = new
        ArrayBlockingQueue<>(1);

    public Escritor(FicheroSimulado f, Controlador c, String n) {
        this.fichero = f;
        this.controlador = c;
        this.nombre = n;
    }
}
```

```
@Override
public void run() {
    try {
        while (true) {
            System.out.println(nombre + " pide permiso para
                escribir...");
            controlador.enviarPetición(new
                PeticiónAbrirEscritura(nombre, miBuzon));
            miBuzon.take();
            System.out.println(nombre + " obtuvo permiso.");
            fichero.escribir(nombre);
            controlador.enviarPetición(new
                PeticiónCerrarEscritura(nombre));
            Thread.sleep((long) (Math.random() * 3000));
        }
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}

public class PmAle {
    public static void main(String[] args) {
        FicheroSimulado fichero = new FicheroSimulado();
        Controlador controlador = new Controlador();
        System.out.println("Iniciando Simulación (Paso de Mensajes -
            Gestor de Bloqueos)");
        new Escritor(fichero, controlador, "Escritor A").start();
        new Escritor(fichero, controlador, "Escritor B").start();
        new Lector(fichero, controlador, "Lector 1").start();
        new Lector(fichero, controlador, "Lector 2").start();
        new Lector(fichero, controlador, "Lector 3").start();
        new Lector(fichero, controlador, "Lector 4").start();
    }
}
```

6.9. Paso de Mensajes Asíncrono: Problema de la cena de los filósofos

Este ejercicio nos propone resolver el problema clásico de la cena de los filósofos mediante paso de mensajes asíncronos. En este caso vamos a plantearlo como sigue. Cinco filósofos ($N=5$) se sientan alrededor de una mesa circular. Entre cada par de filósofos hay un palillo (tenedor). Por lo tanto, hay 5 palillos. Cada filósofo alterna entre dos estados: pensar y comer. Para comer, un filósofo necesita coger los dos palillos que tiene a su lado (el izquierdo y el derecho). Un filósofo solo puede coger un palillo a la vez. En este caso, lo hemos implementado utilizando la técnica de **adquisición atómica** [21][18], en contraste con los semáforos y monitores que usamos la técnica del **camarero o árbitro**.

```
// fichero PmAcf.java

import java.util.Random;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.Queue;
import java.util.LinkedList;

/** Interfaz base para los mensajes */
interface Mensaje {}

class PeticionCoger implements Mensaje {
    public final int idFilosofo;
    public final BlockingQueue<Boolean> canalRespuesta;

    public PeticionCoger(int id, BlockingQueue<Boolean> canal) {
        this.idFilosofo = id;
        this.canalRespuesta = canal;
    }
}

class PeticionSoltar implements Mensaje {
    public final int idFilosofo;
    public PeticionSoltar(int id) {
        this.idFilosofo = id;
    }
}
```

```
}

class MesaActor extends Thread {

    public static final int N_FILOSOFOS = 5;
    private final BlockingQueue<Mensaje> buzonPeticiones = new
        LinkedBlockingQueue<>();
    private final boolean[] palillosLibres = new boolean[N_FILOSOFOS];
    private final Queue<PeticionCoger> filosofosEnEspera = new
        LinkedList<>();

    public MesaActor() {
        for (int i = 0; i < N_FILOSOFOS; i++) {
            palillosLibres[i] = true;
        }
        this.start();
    }

    public void enviarPeticion(Mensaje msg) throws
        InterruptedException {
        this.buzonPeticiones.put(msg);
    }

    @Override
    public void run() {
        System.out.println("Mesa (Servidor) iniciada. Esperando
            peticiones...");
        try {
            while (true) {
                Mensaje msg = buzonPeticiones.take();
                if (msg instanceof PeticionCoger) {
                    procesarPeticionCoger((PeticionCoger) msg);
                }
                else if (msg instanceof PeticionSoltar) {
                    procesarPeticionSoltar((PeticionSoltar) msg);
                }
            }
        } catch (InterruptedException e) {
            System.out.println("Mesa (Servidor) interrumpida.");
        }
    }
}
```

6.9. Paso de Mensajes Asíncrono: Problema de la cena de los filósofos

```
    }  
}  
  
private void procesarPeticionCoger(PeticionCoger p) throws  
    InterruptedException {  
    int id = p.idFilosofo;  
    int izq = id;  
    int der = (id + 1) % N_FILOSOFOS;  
  
    if (palillosLibres[izq] && palillosLibres[der]) {  
        palillosLibres[izq] = false;  
        palillosLibres[der] = false;  
        System.out.println("Mesa: Concediendo palillos a Filósofo  
            " + id);  
        p.canalRespuesta.put(true);  
    } else {  
        // Poner en espera  
        System.out.println("Mesa: Filósofo " + id + " en espera  
            (palillos ocupados).");  
        filosofosEnEspera.add(p);  
    }  
}  
  
private void procesarPeticionSoltar(PeticionSoltar p) throws  
    InterruptedException {  
    int id = p.idFilosofo;  
    int izq = id;  
    int der = (id + 1) % N_FILOSOFOS;  
    palillosLibres[izq] = true;  
    palillosLibres[der] = true;  
    System.out.println("Mesa: Filósofo " + id + " soltó  
        palillos.");  
  
    // Revisar cola de espera  
    int enEsperaSize = filosofosEnEspera.size();  
    for (int i = 0; i < enEsperaSize; i++) {  
        PeticionCoger peticion = filosofosEnEspera.poll();  
  
        int p_id = peticion.idFilosofo;  
        int p_izq = p_id;
```

```
        int p_der = (p_id + 1) % N_FILOSOFOS;

        if (palillosLibres[p_izq] && palillosLibres[p_der]) {
            palillosLibres[p_izq] = false;
            palillosLibres[p_der] = false;
            System.out.println("Mesa: Concediendo palillos a " +
                               p_id + " (estaba en espera).");
            peticion.canalRespuesta.put(true);
        } else {
            filosofosEnEspera.add(peticion);
        }
    }
}

class FilosofoActor extends Thread {

    private final int id;
    private final MesaActor mesa;
    private final Random random = new Random();
    private final BlockingQueue<Boolean> miBuzon = new
        ArrayBlockingQueue<>(1);

    public FilosofoActor(int id, MesaActor mesa) {
        this.id = id;
        this.mesa = mesa;
    }

    private void pensar() throws InterruptedException {
        System.out.println("Filósofo " + id + " está PENSANDO.");
        Thread.sleep(random.nextInt(2000) + 1000);
    }

    private void comer() throws InterruptedException {
        System.out.println("Filósofo " + id + " está COMIENDO.");
        Thread.sleep(random.nextInt(1500) + 1000);
    }

    @Override
    public void run() {
        try {
```

6.9. Paso de Mensajes Asíncrono: Problema de la cena de los filósofos

```
        while (true) {
            pensar();
            System.out.println("Filósofo " + id + " tiene HAMBRE y
                               pide palillos.");
            PeticionCoger peticion = new PeticionCoger(id,
                                                       miBuzon);
            mesa.enviarPeticion(peticion);
            miBuzon.take();
            System.out.println("Filósofo " + id + " obtuvo permiso
                               y palillos.");
            comer();
            System.out.println("Filósofo " + id + " terminó de
                               comer. Suelta palillos.");
            mesa.enviarPeticion(new PeticionSoltar(id));
        }
    } catch (InterruptedException e) {
        System.out.println("Filósofo " + id + " fue
                           interrumpido.");
    }
}

}

public class PmAcf {
    public static void main(String[] args) {
        System.out.println("Iniciando la Cena de los Filósofos (Paso
                           de Mensajes Asíncrono)...");

        // Usamos las clases renombradas
        MesaActor mesa = new MesaActor();
        for (int i = 0; i < MesaActor.N_FILOSOFOS; i++) {
            new FilosofoActor(i, mesa).start();
        }
    }
}
```

6.10. Paso de Mensajes Síncrono: El problema del productor consumidor

Este ejercicio nos propone resolver el problema de los productores-consumidores utilizando paso de mensajes síncronos para acceso concurrente a un mismo buffer compartido circular y sincronización. En este caso utilizaremos `ArrayBlockingQueueE` [16] para trabajar con el paso de mensajes y buzones.

```
// fichero PmSpc.java
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.SynchronousQueue;

//Clase que representa al hilo Productor.Envía "mensajes" (los ítems)
a un buzón síncrono.
class Productor extends Thread {

    private BlockingQueue<Integer> buzon;
    public Productor(BlockingQueue<Integer> buzon) {
        this.buzon = buzon;
    }
    @Override
    public void run() {
        int item = 0;
        try {
            while (true) {
                item++;
                Thread.sleep((int) (Math.random() * 1000));
                System.out.println("Productor produce: " + item + " e
                                INTENTA ENVIAR...");
                buzon.put(item);
                System.out.println("...Productor ENVIÓ " + item + "
                                (El consumidor lo recibió)");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

6.10. Paso de Mensajes Síncrono: El problema del productor consumidor

```
//Clase que representa al hilo Consumidor. Recibe "mensajes" (los í
tems) de un buzón síncrono.
class Consumidor extends Thread {

    private BlockingQueue<Integer> buzon;
    public Consumidor(BlockingQueue<Integer> buzon) {
        this.buzon = buzon;
    }
    @Override
    public void run() {
        try {
            while (true) {
                System.out.println("Consumidor ESPERANDO para
                    recibir...");
                int item = buzon.take();
                System.out.println("Consumidor RECIBIÓ: " + item);
                Thread.sleep((int) (Math.random() * 1500));
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

public class PmSpc {
    public static void main(String[] args) {

        BlockingQueue<Integer> buzon = new SynchronousQueue<>();
        Productor p1 = new Productor(buzon);
        p1.start();
        Consumidor c1 = new Consumidor(buzon);
        c1.start();
        Productor p2 = new Productor(buzon);
        p2.start();
        Consumidor c2 = new Consumidor(buzon);
        c2.start();
    }
}
```

6.11. Paso de Mensajes Síncrono: Problema de los lectores escritores

Este ejercicio nos propone resolver el problema de los lectores-escritores utilizando paso de mensajes síncronos para acceso concurrente a un mismo buffer compartido circular y sincronización. En este caso utilizaremos `ArrayBlockingQueueE` [16] y prioridad para el escritor (**Algoritmo de Courtois**) [18] para trabajar con el paso de mensajes y buzones.

```
// fichero PmSle.java

import java.util.concurrent.BlockingQueue;
import java.util.concurrent.SynchronousQueue;
import java.util.Queue;
import java.util.LinkedList;

class FicheroSimulado {
    private String contenido = "[Contenido inicial del fichero]";

    public void leer(String nombre) {
        System.out.println(nombre + " está LEYENDO: " +
            this.contenido);
        try {
            Thread.sleep((long) (Math.random() * 500));
        } catch (InterruptedException e) {}
    }

    public void escribir(String nombre) {
        System.out.println(nombre + " está ESCRIBIENDO...");
        try {
            Thread.sleep((long) (Math.random() * 1500));
        } catch (InterruptedException e) {}
        this.contenido = "[Escrito por " + nombre + "]";
        System.out.println("... " + nombre + " terminó de escribir.");
    }
}

interface Mensaje {}
class PeticionAbrirLectura implements Mensaje {
```

6.11. Paso de Mensajes Síncrono: Problema de los lectores escritores

```
public final String nombre;
public final BlockingQueue<Boolean> canalRespuesta;
public PeticionAbrirLectura(String n, BlockingQueue<Boolean> c) {
    nombre = n; canalRespuesta = c; }
}
class PeticionCerrarLectura implements Mensaje {
    public final String nombre;
    public PeticionCerrarLectura(String n) { nombre = n; }
}
class PeticionAbrirEscritura implements Mensaje {
    public final String nombre;
    public final BlockingQueue<Boolean> canalRespuesta;
    public PeticionAbrirEscritura(String n, BlockingQueue<Boolean> c)
        { nombre = n; canalRespuesta = c; }
}
class PeticionCerrarEscritura implements Mensaje {
    public final String nombre;
    public PeticionCerrarEscritura(String n) { nombre = n; }
}

class Controlador extends Thread {

    private final BlockingQueue<Mensaje> buzonPeticiones = new
        SynchronousQueue<>();

    private int lectoresActivos = 0;
    private boolean escritorActivo = false;
    private final Queue<PeticionAbrirLectura> lectoresEnEspera = new
        LinkedList<>();
    private final Queue<PeticionAbrirEscritura> escritoresEnEspera =
        new LinkedList<>();

    public Controlador() {
        this.start();
    }

    public void enviarPeticion(Mensaje msg) throws
```

```
InterruptedException {
    buzonPeticiones.put(msg);
}

@Override
public void run() {
    System.out.println("Controlador de bloqueos (SÍNCRONO)
        iniciado.");
    try {
        while (true) {

            Mensaje msg = buzonPeticiones.take();

            if (msg instanceof PeticionAbrirLectura) {
                procesarAbrirLectura((PeticionAbrirLectura) msg);
            }
            else if (msg instanceof PeticionCerrarLectura) {
                procesarCerrarLectura((PeticionCerrarLectura) msg);
            }
            else if (msg instanceof PeticionAbrirEscritura) {
                procesarAbrirEscritura((PeticionAbrirEscritura)
                    msg);
            }
            else if (msg instanceof PeticionCerrarEscritura) {
                procesarCerrarEscritura((PeticionCerrarEscritura)
                    msg);
            }
        }
    } catch (InterruptedException e) {
        System.out.println("Controlador interrumpido.");
    }
}

private void procesarAbrirLectura(PeticionAbrirLectura p) throws
    InterruptedException {
    if (escritoresEnEspera.size() > 0 || escritorActivo) {
        System.out.println("    Controlador: " + p.nombre + " en
```

6.11. Paso de Mensajes Síncrono: Problema de los lectores escritores

```
        espera (Prioridad Escritor).");
        lectoresEnEspera.add(p);
    } else {
        lectoresActivos++;
        p.canalRespuesta.put(true);
    }
}

private void procesarCerrarLectura(PeticionCerrarLectura p) throws
    InterruptedException {
    lectoresActivos--;
    if (lectoresActivos == 0) {
        intentarDespertarEscritor();
    }
}

private void procesarAbrirEscritura(PeticionAbrirEscritura p)
    throws InterruptedException {
    if (lectoresActivos > 0 || escritorActivo) {
        System.out.println("    Controlador: " + p.nombre + " en
            espera (Recurso ocupado).");
        escritoresEnEspera.add(p);
    } else {
        escritorActivo = true;
        p.canalRespuesta.put(true);
    }
}

private void procesarCerrarEscritura(PeticionCerrarEscritura p)
    throws InterruptedException {
    escritorActivo = false;
    if (!intentarDespertarEscritor()) {
        intentarDespertarLectores();
    }
}

private boolean intentarDespertarEscritor() throws
    InterruptedException {
    if (escritoresEnEspera.size() > 0 && lectoresActivos == 0 &&
        !escritorActivo) {
        PeticionAbrirEscritura p = escritoresEnEspera.remove();
        System.out.println("Controlador: Despertando a escritor ")
```

```
        + p.nombre);
    escritorActivo = true;
    p.canalRespuesta.put(true);
    return true;
}
return false;
}

private void intentarDespertarLectores() throws
InterruptedException {
    if (escriitoresEnEspera.size() == 0 && !escritorActivo) {
        while (!lectoresEnEspera.isEmpty()) {
            PeticionAbrirLectura p = lectoresEnEspera.remove();
            System.out.println("Controlador: Despertando a lector
                               " + p.nombre);
            lectoresActivos++;
            p.canalRespuesta.put(true);
        }
    }
}

// Hilo Lector
class Lector extends Thread {
    private final FicheroSimulado fichero;
    private final Controlador controlador;
    private final String nombre;
    private final BlockingQueue<Boolean> miBuzon = new
        SynchronousQueue<>();

    public Lector(FicheroSimulado f, Controlador c, String n) {
        this.fichero = f;
        this.controlador = c;
        this.nombre = n;
    }

    @Override
    public void run() {
        try {
            while (true) {
                System.out.println(nombre + " pide permiso para
```

6.11. Paso de Mensajes Síncrono: Problema de los lectores escritores

```
        leer...");

        controlador.enviarPetición(new
            PeticiónAbrirLectura(nombre, miBuzon));

        miBuzon.take();

        System.out.println(nombre + " obtuvo permiso.");
        fichero.leer(nombre);

        controlador.enviarPetición(new
            PeticiónCerrarLectura(nombre));

        Thread.sleep((long) (Math.random() * 2000));
    }
} catch (InterruptedException e) {
    e.printStackTrace();
}
}
}

// Hilo Escritor (Cliente)
class Escritor extends Thread {
    private final FicheroSimulado fichero;
    private final Controlador controlador;
    private final String nombre;

    private final BlockingQueue<Boolean> miBuzon = new
        SynchronousQueue<>();

    public Escritor(FicheroSimulado f, Controlador c, String n) {
        this.fichero = f;
        this.controlador = c;
        this.nombre = n;
    }

    @Override
    public void run() {
        try {
            while (true) {
```

```
        System.out.println(nombre + " pide permiso para
            escribir...");

        controlador.enviarPetición(new
            PeticiónAbrirEscritura(nombre, miBuzon));
        miBuzon.take();

        System.out.println(nombre + " obtuvo permiso.");
        fichero.escribir(nombre);

        controlador.enviarPetición(new
            PeticiónCerrarEscritura(nombre));

        Thread.sleep((long) (Math.random() * 3000));
    }
} catch (InterruptedException e) {
    e.printStackTrace();
}
}

}

public class PmSle {

    public static void main(String[] args) {

        FicheroSimulado fichero = new FicheroSimulado();
        Controlador controlador = new Controlador();
        System.out.println("Iniciando Simulación (Paso de Mensajes
            SÍNCRONO)");

        new Escritor(fichero, controlador, "Escritor A").start();
        new Escritor(fichero, controlador, "Escritor B").start();
        new Lector(fichero, controlador, "Lector 1").start();
        new Lector(fichero, controlador, "Lector 2").start();
        new Lector(fichero, controlador, "Lector 3").start();
        new Lector(fichero, controlador, "Lector 4").start();

    }
}
```

6.12. Paso de Mensajes Síncrono: Problema de la cena de los filósofos

Este ejercicio nos propone resolver el problema clásico de la cena de los filósofos mediante paso de mensajes síncronos. En este caso vamos a plantearlo como sigue. Cinco filósofos ($N=5$) se sientan alrededor de una mesa circular. Entre cada par de filósofos hay un palillo (tenedor). Por lo tanto, hay 5 palillos. Cada filósofo alterna entre dos estados: pensar y comer. Para comer, un filósofo necesita coger los dos palillos que tiene a su lado (el izquierdo y el derecho). Un filósofo solo puede coger un palillo a la vez. En este caso, lo hemos implementado utilizando la técnica de **adquisición atómica** [21][18], en contraste con los semáforos y monitores que usamos la técnica del **camarero o árbitro**.

```
// fichero PmScf.java
import java.util.Random;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.SynchronousQueue;
import java.util.Queue;
import java.util.LinkedList;

/** Interfaz base para los mensajes */
interface Mensaje {}

class PeticionCoger implements Mensaje {
    public final int idFilosofo;
    public final BlockingQueue<Boolean> canalRespuesta;

    public PeticionCoger(int id, BlockingQueue<Boolean> canal) {
        this.idFilosofo = id;
        this.canalRespuesta = canal;
    }
}

class PeticionSoltar implements Mensaje {
    public final int idFilosofo;
    public PeticionSoltar(int id) {
        this.idFilosofo = id;
    }
}

/**
```

```
* La clase Mesa es un hilo servidor activo.
* Gestiona el estado de los palillos y las peticiones SÍNCRONAS.*/
class Mesa extends Thread {

    public static final int N_FILOSOFOS = 5;

    private final BlockingQueue<Mensaje> buzonPeticiones = new
        SynchronousQueue<>();

    private final boolean[] palillosLibres = new boolean[N_FILOSOFOS];
    private final Queue<PeticionCoger> filosofosEnEspera = new
        LinkedList<>();

    public Mesa() {
        for (int i = 0; i < N_FILOSOFOS; i++) {
            palillosLibres[i] = true;
        }
        this.start();
    }

    public void enviarPeticion(Mensaje msg) throws
        InterruptedException {
        this.buzonPeticiones.put(msg);
    }

    @Override
    public void run() {
        System.out.println("Mesa (Servidor SÍNCRONO) iniciada.
            Esperando peticiones...");
        try {
            while (true) {
                Mensaje msg = buzonPeticiones.take();

                if (msg instanceof PeticionCoger) {
                    procesarPeticionCoger((PeticionCoger) msg);
                }
                else if (msg instanceof PeticionSoltar) {
                    procesarPeticionSoltar((PeticionSoltar) msg);
                }
            }
        }
    }
}
```

6.12. Paso de Mensajes Síncrono: Problema de la cena de los filósofos

```
    }
} catch (InterruptedException e) {
    System.out.println("Mesa (Servidor) interrumpida.");
}
}

private void procesarPeticionCoger(PeticionCoger p) throws
InterruptedException {
    int id = p.idFilosofo;
    int izq = id;
    int der = (id + 1) % N_FILOSOFOS;

    if (palillosLibres[izq] && palillosLibres[der]) {
        palillosLibres[izq] = false;
        palillosLibres[der] = false;
        System.out.println("Mesa: Concediendo palillos a Filósofo
            " + id);
        p.canalRespuesta.put(true);
    } else {
        System.out.println("Mesa: Filósofo " + id + " en espera
            (palillos ocupados).");
        filosofosEnEspera.add(p);
    }
}

private void procesarPeticionSoltar(PeticionSoltar p) throws
InterruptedException {
    int id = p.idFilosofo;
    int izq = id;
    int der = (id + 1) % N_FILOSOFOS;

    palillosLibres[izq] = true;
    palillosLibres[der] = true;
    System.out.println("Mesa: Filósofo " + id + " soltó
        palillos.");

    int enEsperaSize = filosofosEnEspera.size();
    for (int i = 0; i < enEsperaSize; i++) {
        PeticionCoger peticion = filosofosEnEspera.poll();
```

```
int p_id = peticion.idFilosofo;
int p_izq = p_id;
int p_der = (p_id + 1) % N_FILOSOFOS;

if (palillosLibres[p_izq] && palillosLibres[p_der]) {
    palillosLibres[p_izq] = false;
    palillosLibres[p_der] = false;
    System.out.println("Mesa: Concediendo palillos a " +
        p_id + " (estaba en espera).");
    peticion.canalRespuesta.put(true);
    break;
} else {
    filosofosEnEspera.add(peticion);
}
}
}

/**
 * El hilo Filosofo (Cliente).*/
class Filosofo extends Thread {

    private final int id;
    private final Mesa mesa;
    private final Random random = new Random();

    private final BlockingQueue<Boolean> miBuzon = new
        SynchronousQueue<>();

    public Filosofo(int id, Mesa mesa) {
        this.id = id;
        this.mesa = mesa;
    }

    private void pensar() throws InterruptedException {
        System.out.println("Filósofo " + id + " está PENSANDO.");
        Thread.sleep(random.nextInt(2000) + 1000);
    }

    private void comer() throws InterruptedException {
        System.out.println("Filósofo " + id + " está COMIENDO.");
    }
}
```

6.12. Paso de Mensajes Síncrono: Problema de la cena de los filósofos

```
        Thread.sleep(random.nextInt(1500) + 1000);
    }
    @Override
    public void run() {
        try {
            while (true) {
                pensar();
                System.out.println("    Filósofo " + id + " tiene
                    HAMBRE y pide palillos.");
                PeticionCoger peticion = new PeticionCoger(id,
                    miBuzon);
                mesa.enviarPeticion(peticion);
                miBuzon.take();
                System.out.println("Filósofo " + id + " obtuvo permiso
                    y palillos.");
                comer();
                System.out.println("Filósofo " + id + " termin? de
                    comer. Suelta palillos.");
                mesa.enviarPeticion(new PeticionSoltar(id));
            }
        } catch (InterruptedException e) {
            System.out.println("Filósofo " + id + " fue
                interrumpido.");
        }
    }
}

public class PmScf {
    public static void main(String[] args) {
        System.out.println("Iniciando la Cena de los Filósofos (Paso
            de Mensajes SÍNCRONO)...");
        Mesa mesa = new Mesa();
        for (int i = 0; i < Mesa.N_FILOSOFOS; i++) {
            new Filosofo(i, mesa).start();
        }
    }
}
```


Bibliografía

- [1] J. Gosling B. Joy, G. Steele. *The Java Language Specification*. Addison-Wesley, first edition, 1996.
- [2] Merino L. Santos E. *Álgebra Lineal con métodos elementales*. Paraninfo, 1997.
- [3] J. Gosling H. McGilton. *The Java Language Environment: A White Paper*. Sun Microsystems, first edition, 1995.
- [4] Stewart James. *Cálculo - Trascendentes Tempranas*. CENGAGE, eight edition, 2018.
- [5] JetBrains. *IntelliJIDEA* <https://www.jetbrains.com/idea/>. 2025.
- [6] Bloch Joshua. *Effective Java*. Addison-Wesley, third edition, 2017.
- [7] J. Gosling K. Arnold. *The Java Programming Language*. Addison-Wesley, first edition, 1996.
- [8] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley Professional, second edition, 1998.
- [9] Lucena López Manuel. *Criptografía y Seguridad en computadores*. Universidad de Jaén, fifth edition, 2023.
- [10] Microsoft. *Visual Studio Code* <https://code.visualstudio.com/>. 2025.
- [11] OpenJDK. *OpenJDK 21 LTS* <https://openjdk.org/projects/jdk/21/>. 2023.
- [12] OpenJDK. *OpenJDK 25 LTS* <https://openjdk.org/projects/jdk/25/>. 2025.
- [13] G. Brassard P. Bratley. *Fundamentos de algoritmia*. Prentice Hall., 1998.
- [14] F.M Pérez Montes. *Ejercicios de Programación en Java Condicionales, Bucles, Tablas y Funciones*. Asoc. Por la Innovación Educativa, first edition, 2001.
- [15] Johnson Richard. *Sorting Algorithms and Techniques: Definitive Referente for Developers and Engineers*. AV, first edition, 2025.
- [16] Herber Schildt. *Java: The Complete Reference*. McGraw Hill, thirteen edition, 2024.

- [17] Wild Sebastian. *Java 7s Dual-Pivot Quicksort*. AV, first edition, 2014.
- [18] Andrew S. Tanenbaum and Herbert Bos. *Sistemas operativos modernos*. Pearson, 4 edition, 2016.
- [19] Kahan W. *Miscalculating Area and Angles of Needle-like Triangle*. Berkeley, <https://people.eecs.berkeley.edu/~wkahan/triangle.pdf> edition, 2014.
- [20] Mark Allen Weiss. *Data Structures and Algorithm Analysis in C*. Person, second edition, 2003.
- [21] Palma Méndez José Tomás y otros. *Programación Concurrente*. Thomson, first edition, 2006.